Java para Internet

VAGNER ERENO QUINCOZES

unicesumar

EDIÇÃO 01

EXPEDIENTE

Equipe Multidisciplinar da Pós-Graduação EAD

Victor Vinicius Biazon
Katia C. Cavalcante De Oliveira
Rodislaine Adriana Lourenco Pimenta
Anderson P. do Amaral
Michele de Almeida Pereira
Liana Gomes Netto
Fernanda Sutkus de Oliveira Mello
Alana Beatriz Lemos Ribeiro Longhi
Flávia Thais Pedroso

FICHA CATALOGRÁFICA

N964 Núcleo de Educação a Distância. **QUINCOZES**, Vagner Ereno.

Java para Internet / Vagner Ereno Quincozes. - Florianópolis, SC: Arqué, 2025.

108 p.

ISBN digital 978-65-5466-718-0

1. Java 2. Sistemas 3. Internet 4. EaD. I. Título.

CDD - 005.133

Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Ano de impressão:

Impresso por:

RECURSOS DE IMERSÃO

PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.

APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.

EU INDICO

Utilizado para agregar um conteúdo externo.

EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.

ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.

INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.

SUMÁRIO

5

UNIDADE 1

FUNDAMENTOS DA WEB E INTEGRAÇÃO COM BANCO DE DADOS 5

41

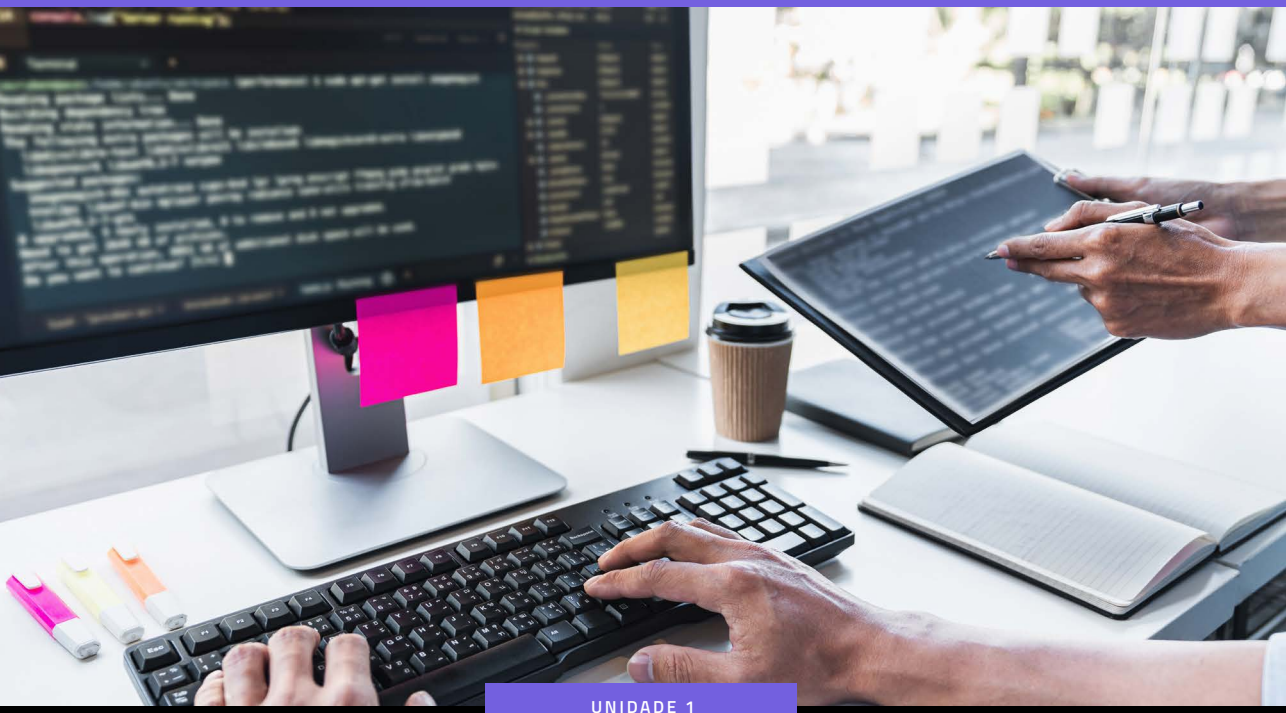
UNIDADE 2

JAVA SERVLET E JAVASERVER PAGES (JSP) 41

74

UNIDADE 3

JAVASERVER FACES E PADRÕES DE PROJETO MVC 74



UNIDADE 1

FUNDAMENTOS DA WEB E INTEGRAÇÃO COM BANCO DE DADOS

MINHAS METAS

- Compreender o modelo cliente-servidor e o fluxo HTTP.
- Construir formulários HTML para envio de dados ao servidor.
- Aplicar comandos SQL para manipular dados em banco relacional.
- Integrar aplicações Java a bancos usando JDBC com segurança.

INICIE SUA JORNADA

Em um hospital de médio porte, a recepção ainda trabalha com fichas de papel. A cada novo atendimento, dados são preenchidos manualmente, arquivados fisicamente e resgatados com dificuldade sempre que necessário. Enquanto isso, a equipe médica enfrenta atrasos, a fila cresce, e os erros se acumulam. Situações como essa continuam sendo realidade em muitos setores — e a falta de sistemas web eficientes está no centro desse problema.

Soluções digitais capazes de automatizar e organizar processos como esse dependem diretamente de duas engrenagens principais: a estrutura da Web e a integração com banco de dados. Compreender como uma requisição sai do navegador, chega ao servidor, é processada por um código Java e resulta em uma alteração segura nos dados armazenados é essencial para quem deseja atuar no desenvolvimento *backend*. Essa compreensão permite construir sistemas mais ágeis, confiáveis e escaláveis. Essas qualidades são cada vez mais exigidas pelo mercado.

Ao longo deste tema, você terá a oportunidade de desenvolver uma aplicação Java real, que se conecta a um banco de dados MySQL para realizar operações de cadastro e consulta. Vai entender como uma URL carrega dados, programar conexões com JDBC e utilizar SQL para gerenciar informações. Tudo isso com exemplos, em um ambiente próximo à realidade profissional.

Ao caminhar por este tema, vale refletir: como o domínio desses fundamentos pode transformar a forma como você propõe soluções? O que significa, de fato, desenvolver para a Web em um mundo onde os dados são ativos valiosos e o tempo de resposta é decisivo? Pensar sobre isso é mais do que um exercício técnico, é um passo importante para se tornar um desenvolvedor estratégico, que alia conhecimento técnico à visão de impacto social e profissional.

PLAY NO CONHECIMENTO

Quer entender como organizar o acesso ao banco de dados de forma mais profissional? Dê o play no nosso podcast "DAO na prática: organizando o acesso a dados em aplicações Java" e descubra como esse padrão pode transformar seu código, torná-lo mais limpo, modular e preparado para crescer.

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À WEB E AO MODELO CLIENTE-SERVIDOR

Você já acessou um site hoje? Talvez tenha feito uma compra on-line ou lido uma notícia. Por trás dessas ações simples está um modelo arquitetural que permite que tudo funcione em tempo real: o modelo **cliente-servidor**. Essa é a base da Web, e para desenvolver aplicações Java para internet, é fundamental compreendê-la com clareza (Kurose; Ross, 2020).

No modelo cliente-servidor, temos dois protagonistas: o **cliente**, geralmente um navegador como o Chrome, e o **servidor**, que processa as requisições e envia as respostas. O cliente é responsável por enviar pedidos, como "quero ver a página inicial do site", e o servidor responde, enviando o conteúdo correspondente, como arquivos HTML, imagens ou dados estruturados.

Para que esse processo funcione, é necessário um **servidor web**, como o Apache ou o Tomcat. Esse tipo de servidor escuta as requisições que chegam pela internet, interpreta o que o cliente está pedindo e encaminha a resposta adequada. Quando desenvolvemos com Java, geralmente usamos o Tomcat como container de *servlets* — ele recebe as requisições HTTP e executa nossas classes Java que tratam essas requisições.

A figura a seguir ilustra bem o conceito do modelo Cliente-Servidor.

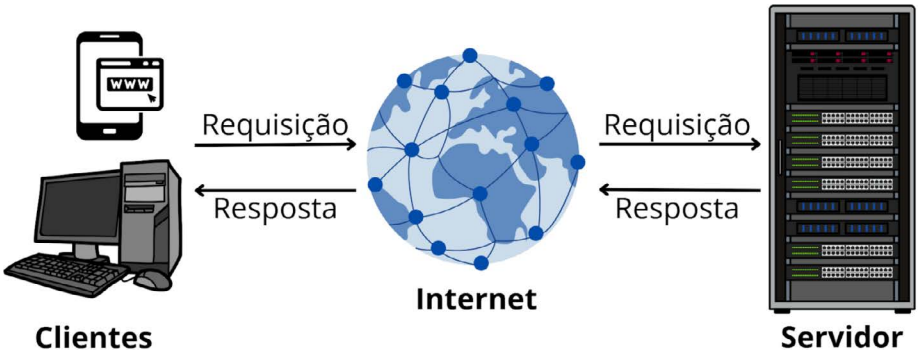


Figura 1 - Modelo Cliente-Servidor / Fonte: O autor.

Descrição da Imagem: à esquerda, há dois dispositivos representando os clientes: um computador e um celular. Ao centro, há um globo azul representando a internet. À direita, há um grande servidor com vários componentes eletrônicos representando o servidor. Setas indicam o fluxo de dados: dos clientes para o servidor com a palavra "Requisição", e do servidor de volta aos clientes com a palavra "Resposta". Fim da descrição.

Para e–tender melhor, vamos visualizao um exemplo: imagine que você digita www.livrariaonline.com no navegador. O que acontece é o seguinte (Kurose; Ross, 2020):

CLIENTE ENVIA A REQUISIÇÃO

O cliente (navegador) monta uma requisição HTTP para acessar o site.

INTERNET ENTREGA AO SERVIDOR

Essa requisição é enviada pela internet até o servidor da livraria.

SERVIDOR PROCESSA A REQUISIÇÃO

O servidor web (ex: Tomcat) interpreta a requisição e chama uma aplicação Java responsável por montar a página.

SERVIDOR DEVOLVE A RESPOSTA
O conteúdo gerado (HTML, imagens etc.) é enviado de volta ao cliente.
NAVEGADOR EXIBE O CONTEÚDO
O navegador exibe o conteúdo na tela.

Esse ciclo de requisição e resposta ocorre em segundos, mas depende de cada um desses componentes funcionando corretamente. E o papel do desenvolvedor *backend* é programar o lado do servidor para que ele saiba como responder a cada requisição com os dados certos, no formato correto.

O modelo de Requisição e Resposta HTTP

Quando um navegador acessa um site, ele está enviando uma requisição para algum servidor. Esse pedido é feito através do protocolo HTTP, sigla para *Hyper-Text Transfer Protocol*. Ele define as regras de comunicação entre cliente e servidor na Web, sendo um dos pilares da internet como conhecemos (Kurose e Ross, 2020).

Toda vez que acessamos uma página, preenchemos um formulário ou clicamos em um botão, uma **requisição HTTP** é disparada. Essa requisição contém informações organizadas em partes. As mais importantes são:

MÉTODO HTTP
O método HTTP, que indica o tipo de ação desejada (como GET ou POST).
URL
A URL (*Uniform Resource Locator*), que aponta para o recurso solicitado.

CABEÇALHOS (OU *HEADERS*)
Os cabeçalhos (*headers*), que informam detalhes sobre o navegador, idioma, autenticação e tipo de conteúdo.
CORPO (OU BODY)
Um corpo (body), que aparece geralmente em métodos como POST e carrega dados, como os de um formulário preenchido.

Ao receber uma requisição, o servidor responde com uma resposta HTTP composta por três partes: um **código de status** (como 200, 404 ou 500), que indica o resultado da operação; **cabeçalhos**, com informações como o tipo de conteúdo (text/html, por exemplo); e o **corpo da resposta**, que traz o conteúdo em si — geralmente em HTML, JSON ou outro formato (Kurose; Ross, 2020).

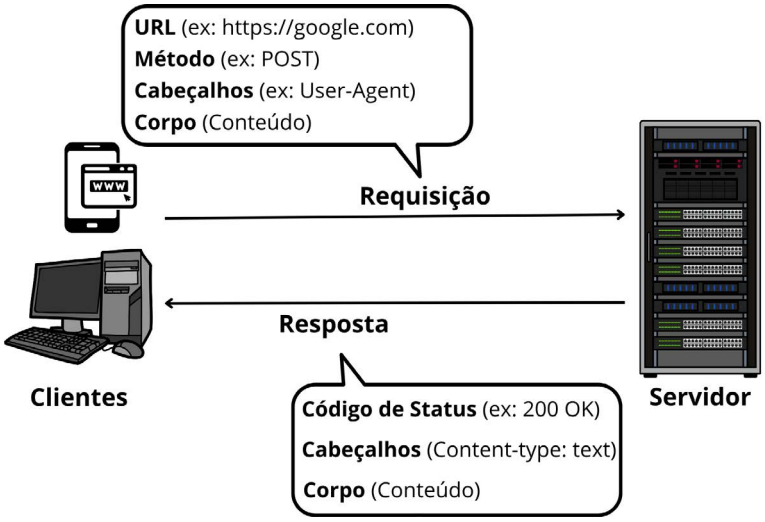


Figura 2 – Componentes do modelo de Requisição e Resposta. / Fonte: o autor.

Descrição da Imagem: diagrama mostra a comunicação entre clientes e servidor via internet. À esquerda, há ícones representando os clientes. À direita, um servidor. Setas indicam o fluxo da requisição (do cliente para o servidor) e da resposta (do servidor para o cliente). A requisição inclui URL, método, cabeçalhos e corpo. A resposta traz um código de status, cabeçalhos e corpo com o conteúdo retornado. Fim da descrição.

A Figura 2 ilustra os componentes típicos do modelo de requisição e resposta.

URLs, métodos HTTP e códigos de *status*

A URL que digitamos no navegador é muito mais do que um endereço. Ela representa o caminho até um recurso na Web e segue uma estrutura bem definida (Kuros; Ross, 2020). Compreender essa estrutura é essencial para construir aplicações Java que saibam interpretar o que o cliente está solicitando. Uma URL (*Uniform Resource Locator*) pode ser dividida em partes, confira a seguir:

https://www.exemplo.com:443/produtos?categoria=livros

1. A primeira parte, https, indica o **protocolo** utilizado para a comunicação, que pode ser http ou https (versão segura).
2. Em seguida, temos www.exemplo.com, que é o **domínio**, ou seja, o nome que identifica o servidor que hospeda o site.
3. Logo após, :443 representa a **porta** utilizada pela aplicação — neste caso, a 443, que é a padrão para conexões seguras, motivo pelo qual muitas vezes ela nem aparece.
4. O trecho /produtos corresponde ao **caminho**, que indica qual recurso está sendo solicitado dentro do servidor.
5. Por fim, ?categoria=livros mostra os **parâmetros** enviados na URL, no formato chave=valor, que servem para transmitir informações adicionais à aplicação, como neste caso, a categoria desejada.

Ao lado da URL, vem o método HTTP, que informa ao servidor qual ação se espera. Os métodos mais utilizados são: o **GET**, que serve para recuperar dados, como quando acessamos uma página da web ou buscamos uma lista de produtos; o **POST**, usado para enviar dados ao servidor, por exemplo, ao submeter um formulário de cadastro; o **PUT**, geralmente utilizado para atualizar um recurso já existente; e o **DELETE**, que tem a função de remover um recurso do servidor. Esses métodos ajudam a definir, de forma clara e padronizada, a intenção de cada requisição dentro da comunicação entre cliente e servidor.

Cada requisição retorna um código de status HTTP que informa o resultado da operação. Os mais comuns são: 200 (OK), quando a requisição foi bem-su-

cedida; 201 (*Created*), usado quando um novo recurso é criado, geralmente em operações POST; 400 (*Bad Request*), quando há erro na formação da requisição; 401 (*Unauthorized*), se o usuário não estiver autenticado; 404 (*Not Found*), quando o recurso não é encontrado; e 500 (*Internal Server Error*), indicando falha no servidor (Kurose; Ross, 2020).

EU INDICO

Quer conhecer a lista completa dos códigos de status HTTP, com explicações simples e exemplos? Acesse aqui o site e explore cada código de forma didática e organizada. Ideal para revisar ou consultar rapidamente durante o desenvolvimento.

Compreender a estrutura da URL, os métodos e os códigos de status (*status codes*) é essencial para escrever código Java que interprete corretamente os pedidos do cliente e retorne respostas úteis, dentro do padrão da Web.

ESTRUTURA BÁSICA DE UM DOCUMENTO HTML

Antes mesmo de o servidor entrar em ação, é o navegador que dá o primeiro passo ao interpretar o que vem do lado do cliente. Essa interface, que é visível para o usuário, é construída em sua maioria com HTML — a linguagem de marcação padrão da Web. HTML (*HyperText Markup Language*) não é uma linguagem de programação no sentido tradicional, mas sim uma linguagem de marcação que define a estrutura e o conteúdo de uma página (Wolf, 2023).

EU INDICO

Descubra, com Motion Graphics, como Java, HTML e JDBC se conectam na construção de aplicações web! Assista e entenda visualmente essa integração essencial para o desenvolvimento web.

Todo documento HTML possui uma estrutura básica:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Minha Página</title>
  </head>
  <body>
    <h1>Bem-vindo</h1>
    <p>Este é um exemplo simples de estrutura HTML.</p>
  </body>
</html>
```

Esse pequeno exemplo já ilustra a maioria dos elementos essenciais. O <!DOCTYPE html> indica que o documento segue o padrão HTML5. As tags <html>, <head> e <body> organizam o conteúdo em blocos. O <head> contém informações sobre a página, como codificação e título, enquanto o <body> apresenta o conteúdo visível (Wolf, 2023).

A grande vantagem do HTML é a sua capacidade de estruturar visualmente as informações. Elementos como títulos, listas, imagens, formulários e botões ajudam o usuário a interagir com os dados e, mais importante ainda, servem como pontos de entrada para a comunicação com o *backend* Java.

Por exemplo, quando criamos um botão "Cadastrar" em HTML, o que acontece ao clicar nesse botão dependerá de como o formulário foi construído e para onde ele aponta. Esse comportamento envolve tanto o HTML quanto o protocolo HTTP, como vimos anteriormente.

Um bom desenvolvedor *backend* não precisa dominar design, mas precisa compreender bem como as páginas HTML se estruturam. Afinal, você será responsável por processar os dados que chegam do cliente e devolver respostas coerentes com a experiência de quem está do outro lado da tela.

Formulários HTML e envio de dados

Um dos recursos mais importantes do HTML para quem desenvolve aplicações web é o **formulário**. É por meio dele que os usuários interagem com o sistema, enviando informações que serão processadas no servidor. Em uma aplicação Java, isso significa que os dados inseridos em um formulário serão recebidos por um Servlet, tratados e, muitas vezes, armazenados em um banco de dados. A estrutura básica de um formulário HTML é simples. Veja um exemplo:

```
<form action="/cadastrar" method="post">
  <label for="nome">Nome:</label>
  <input type="text" id="nome" name="nome">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email">

  <button type="submit">Enviar</button>
</form>
```

Nesse exemplo, o atributo *action* indica para **onde** os dados são enviados. Já o *method* define **como** eles serão enviados. Se for GET, os dados vão na URL. Se for POST, eles são enviados no corpo da requisição (Wolf, 2023).

Cada *input* representa um campo de preenchimento. O atributo *name* define o nome da variável que será recebida do lado do servidor. Em aplicações Java com Servlets, podemos acessar esse valor usando (Oracle, 2020):

```
String nome = request.getParameter("nome");
```

Isso torna o formulário o principal ponto de comunicação entre o cliente (navegador) e o *backend* em Java. Por isso, é fundamental saber como estruturá-los corretamente, garantindo que os dados cheguem com clareza e possam ser processados de forma segura e eficiente.

Além dos campos de texto, formulários podem conter caixas de seleção, campos ocultos, senhas, botões de envio e até arquivos para *upload* (Wolf, 2023). O HTML oferece um conjunto de elementos para montar interfaces de entrada de dados. O desenvolvedor *backend* deve entender como esses dados são organizados e enviados para tratá-los corretamente na aplicação.

Cabeçalhos HTTP e tipos de conteúdo

Quando um formulário é enviado ou uma requisição é feita pelo navegador, não é apenas o conteúdo da mensagem que importa. Os cabeçalhos HTTP desempenham um papel crucial na comunicação entre cliente e servidor (Kurose; Ross, 2020). Eles fornecem informações que ajudam o servidor a entender melhor o que está chegando e como deve responder.

Um dos cabeçalhos mais importantes é o ***Content-Type***. Ele indica o formato dos dados enviados na requisição ou retornados na resposta. Por exemplo, ao enviar um formulário, o navegador geralmente utiliza:

```
Content-Type: application/x-www-form-urlencoded
```

Esse tipo de conteúdo significa que os dados foram codificados no formato padrão de formulários HTML, como nome=Maria&email=maria@example.com. Esse é o formato que o Java está acostumado a processar usando o método request.getParameter(). Existem outros tipos de conteúdo muito comuns no desenvolvimento web (Kurose; Ross, 2020):

APPLICATION/JSON
Para envio de dados em formato JSON, muito utilizado em APIs.
MULTIPART/FORM-DATA
Para envio de arquivos junto com campos de formulário.

TEXT/HTML
Para respostas que contêm páginas web.
APPLICATION/XML
Em sistemas que usam dados em XML.

Imagine que sua aplicação Java receba uma requisição com *Content-Type: application/json*. Nesse caso, os dados não estarão disponíveis por getParameter() diretamente. Você precisará ler o corpo da requisição e usar uma biblioteca como o Jackson ou Gson para interpretar o JSON.

Os cabeçalhos também controlam a resposta (Kurose and Ross, 2020). O servidor pode indicar, por exemplo: Content-Type: text/html. Isso sinaliza ao navegador que a resposta deve ser interpretada como uma página HTML. Se a resposta for um arquivo para download, o servidor pode usar application/octet-stream, e o navegador oferecerá o arquivo ao usuário.

Saber configurar e interpretar os cabeçalhos garante que os dados trafeguem da forma esperada e que o servidor possa se adaptar às diferentes necessidades da aplicação. Cabeçalhos mal definidos podem gerar falhas de leitura, comportamentos inesperados ou até mesmo problemas de segurança.

BANCO DE DADOS RELACIONAL

Aplicações web raramente funcionam sozinhas. Por trás da interface que o usuário vê, há quase sempre um **banco de dados** organizando, armazenando e protegendo informações. Para isso, utilizamos os bancos de dados relacionais (Hernandez, 2021).

Um banco de dados relacional organiza as informações em **tabelas**, que são semelhantes a planilhas. Cada tabela é composta por **linhas** (também chamadas de registros) e **colunas** (que representam os atributos de cada item armazenado) (Hernandez, 2021). Por exemplo, uma tabela chamada usuários pode ter as colunas id, nome, e-mail e senha.

A grande força dos bancos relacionais está na forma como eles conectam diferentes tabelas. Isso é feito por meio de chaves (Hernandez, 2021):

- A **chave primária** (*primary key*) identifica de forma única cada linha em uma tabela. Normalmente, é um identificador único.
- A **chave estrangeira** (*foreign key*) é um campo que faz referência à chave primária de outra tabela, permitindo criar relacionamentos entre elas.

Vamos imaginar um sistema de hospital. Podemos ter uma tabela de pacientes e outra de planos de saúde. Cada paciente está vinculado a um plano, e isso é representado por uma chave estrangeira na tabela de pacientes que aponta para a tabela de planos. Assim, evitamos repetir o nome do plano em cada registro e facilitamos consultas e atualizações no sistema.

Os relacionamentos entre tabelas podem ser: 1) **um para um**: um registro de uma tabela está associado a apenas um de outra; 2) **um para muitos**: um registro de uma tabela pode estar ligado a vários da outra; ou 3) **muitos para muitos**: quando vários registros de uma tabela podem se relacionar com vários da outra (geralmente exige uma terceira tabela para representar a associação).

Compreender essas relações é essencial para projetar estruturas de dados eficientes (Hernandez, 2021), especialmente em aplicações Java que dependem de consistência, integridade e rapidez no acesso às informações.

Linguagem SQL

Depois de entender como os dados se organizam em tabelas e se relacionam por meio de chaves, o próximo passo é aprender a acessar e manipular essas informações. Para isso, utilizamos a **linguagem SQL** (*Structured Query Language*), padrão adotado por praticamente todos os sistemas de banco de dados relacionais, como MySQL, PostgreSQL e Oracle (Hernandez, 2021).

Com SQL, podemos executar operações fundamentais conhecidas como CRUD, sigla para:

Create (Criar): inserir novos dados.

Read (Ler): consultar dados existentes.

Update (Atualizar): atualizar registros.

Delete (Deletar): excluir dados.

A seguir, veremos exemplos básicos de cada uma dessas operações.

Para inserir um novo registro, utilizamos o comando INSERT INTO. Por exemplo, para cadastrar um novo usuário na tabela usuarios, o comando seria: INSERT INTO usuarios (nome, email, senha) VALUES ('Ana Souza', 'ana@email.com', 'senha123');

Já para consultar os dados existentes, usamos o SELECT. É possível buscar todos os campos de todos os usuários com SELECT * FROM usuarios, ou apenas campos específicos como nome e email com SELECT nome, email FROM usuarios. Também é possível filtrar os resultados, como em SELECT * FROM usuarios WHERE nome = 'Ana Souza'; que retorna apenas os registros que atendem à condição indicada.

Quando é necessário alterar um registro, usamos o comando UPDATE. Suponha que o e-mail da usuária com id = 1 tenha mudado. A instrução seria:

```
UPDATE usuarios SET email = 'novoemail@exemplo.com' WHERE id = 1;
```

Por fim, para excluir um dado do banco, aplicamos o comando DELETE. Se quisermos remover o usuário com id = 1, utilizamos:

```
DELETE FROM usuarios WHERE id = 1;
```

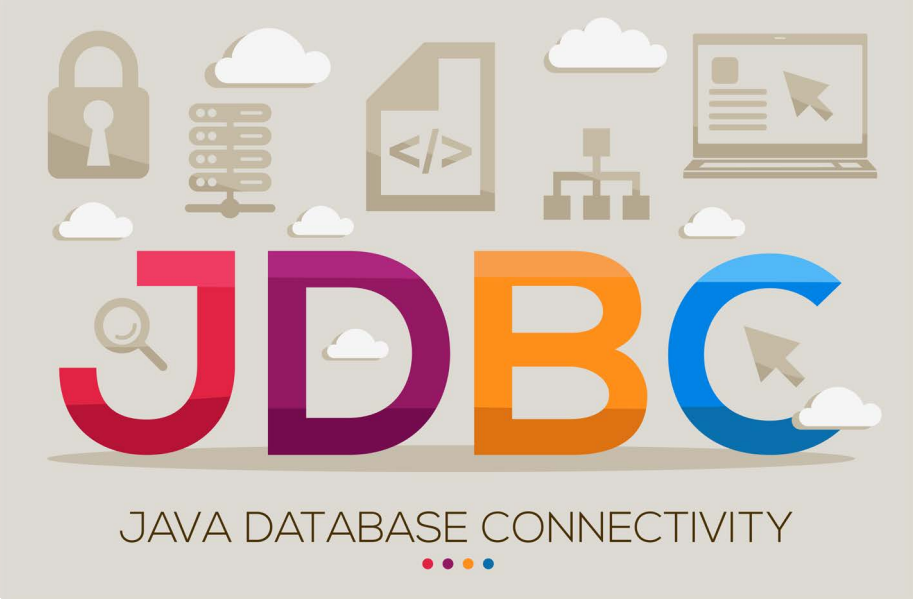
Essas operações representam a base de praticamente todos os sistemas interativos. Formular um bom comando SQL envolve entender a sintaxe e como as tabelas se organizam e se relacionam (Hernandez, 2021).

EU INDICO

Quer aprender ou revisar SQL de forma rápida e prática? O site W3Schools SQL Tutorial oferece explicações simples, exemplos claros e a possibilidade de testar os comandos diretamente no navegador. É uma ótima opção para iniciantes e também para quem deseja reforçar conceitos com agilidade.

INTRODUÇÃO AO JDBC E SEU PAPEL NO JAVA WEB

Agora que entendemos como os dados se organizam no banco, surge a pergunta: como o Java acessa essas informações? A resposta está no **JDBC**, que significa *Java Database Connectivity* (Oracle, 2020). Essa é a API padrão do Java para conectar-se a bancos de dados relacionais, executar comandos SQL e manipular os resultados diretamente a partir da aplicação.



O JDBC atua como uma ponte entre a aplicação Java e o banco de dados. Ele fornece um conjunto de classes e interfaces que tornam possível abrir conexões, executar instruções e recuperar dados, independentemente do sistema de banco utilizado, como MySQL, PostgreSQL ou Oracle. O que muda, nesses casos, é apenas o **driver** específico (Oracle, 2020).

Mesmo com o uso de frameworks modernos como JPA ou Hibernate, o JDBC continua sendo a base da integração entre Java e banco de dados. Compreender seu funcionamento é essencial para quem deseja desenvolver aplicações web sólidas e eficientes.

Etapas de conexão: driver, URL, conexão, *statement*, *ResultSet*

Estabelecer uma conexão entre uma aplicação Java e um banco de dados usando JDBC envolve uma sequência de passos bem definida. Compreender cada etapa é essencial para garantir que a comunicação ocorra de forma eficiente, segura e sem erros (Oracle, 2020). A seguir, explicamos o papel de cada componente envolvido nesse processo.

DRIVER JDBC

O primeiro passo é garantir que o driver JDBC esteja disponível no projeto. Esse driver é um conjunto de classes que entende como conversar com o banco específico que você está usando. Por exemplo, para MySQL, usamos o *mysql-connector-java*, e para PostgreSQL, o *postgresql-42.x.jar*. Em projetos modernos, especialmente com Maven, basta declarar a dependência correta no pom.xml e o driver será adicionado automaticamente.

ESTABELECIMENTO DA CONEXÃO

Com o driver disponível, o código precisa estabelecer a conexão usando o método getConnection da classe DriverManager. Esse método recebe três parâmetros: a URL do banco, o nome de usuário e a senha. A URL contém informações como o tipo de banco, o servidor (host), a porta e o nome da base de dados. Por exemplo:

```
String url = "jdbc:mysql://localhost:3306/sistema";
Connection conexao = DriverManager.getConnection(url, "usuario", "senha");
```

CRIAÇÃO E EXECUÇÃO DE COMANDOS SQL

Após estabelecer a conexão, criamos um *Statement* (ou *PreparedStatement*) para montar e executar instruções SQL. O Statement representa o comando que será enviado ao banco. Por exemplo:

```
Statement stmt = conexao.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM usuarios");
```

Quando usamos comandos como SELECT, os resultados são retornados em um objeto ResultSet, que pode ser percorrido com um laço while.

LEITURA DOS DADOS

A ResultSet permite acessar os dados retornados linha a linha, como se fosse uma tabela navegável. Utilizamos métodos como getString("nome") para ler os valores de cada coluna. Podemos iterar por ele com um laço while, acessando cada coluna por nome ou posição. Veja um exemplo:

```
while (rs.next()) {
    String nome = rs.getString("nome")
    System.out.println("Usuário: " + nome);
}
```

Esse processo é essencial para recuperar e manipular os dados armazenados no banco.

FECHAMENTO DOS RECURSOS

Após o uso, é fundamental fechar o ResultSet, o Statement e a Connection. Veja um exemplo:

```
rs.close();
stmt.close();
conexao.close();
```

Isso evita vazamentos de recursos e sobrecarga no banco, garantindo que a aplicação funcione de forma eficiente e estável.

Essas etapas — carregar o driver, montar a URL, estabelecer a conexão, executar o comando e tratar o resultado — formam a base da integração entre Java e o

banco de dados (Oracle, 2020). Embora simples em conceito, é a partir delas que surgem soluções complexas e profissionais.

Tratamento de exceções e boas práticas com recursos

Quando uma aplicação Java se comunica com um banco de dados por meio do JDBC, diversas coisas podem dar errado. Conexões podem falhar, comandos SQL podem conter erros de sintaxe, e até mesmo o banco pode ficar indisponível. Por isso, é fundamental que você esteja preparado para lidar com exceções e utilize os recursos do Java de forma segura e eficiente.

O primeiro cuidado é sempre envolver as operações com banco de dados dentro de um bloco **try-catch**. O JDBC pode lançar exceções do tipo SQLException, que devem ser capturadas para que a aplicação não seja interrompida de forma inesperada (Oracle, 2020).

Porém, existe uma forma mais moderna e segura de fazer isso em Java: utilizando o **try-with-resources**. Esse recurso garante que os objetos que implementam a interface *AutoCloseable*, como *Connection*, *Statement* e *ResultSet*, sejam automaticamente fechados ao final do bloco try, mesmo que ocorra um erro no meio da execução. Isso reduz o risco de esquecer de fechar recursos, melhora a legibilidade e torna o código mais robusto (Oracle, 2020).

Além disso, sempre que possível, evite construir comandos SQL com concatenação de *strings*. Isso pode abrir brechas para ataques de **injeção de SQL.** O ideal é usar *PreparedStatement*, que veremos mais adiante.

Outra boa prática é isolar a lógica de acesso ao banco de dados em classes específicas. Isso facilita a manutenção e permite que seu código siga os princípios de organização, como a separação de responsabilidades.

CRIANDO SUA PRIMEIRA APLICAÇÃO JAVA COM BANCO DE DADOS

Agora que já entendemos os conceitos fundamentais sobre JDBC e a comunicação entre aplicações Java e bancos de dados, neste capítulo você será guiado na construção de uma aplicação Java real, que se conecta a um banco de dados

MySQL para realizar operações de cadastro e listagem de usuários. Essa atividade reunirá tudo o que você aprendeu até aqui, permitindo visualizar a aplicação prática dos conceitos em um projeto funcional.

Para isso, utilizaremos o ambiente de desenvolvimento **IntelliJ IDEA** e o **Maven** como gerenciador de dependências. Se você ainda não conhece essas ferramentas, não se preocupe: explicaremos o necessário para que você possa acompanhar e reproduzir o projeto em sua própria máquina.

Preparando o ambiente: IntelliJ, MySQL e dependências

Antes de escrevermos qualquer linha de código, precisamos garantir que nosso ambiente de desenvolvimento esteja pronto para rodar uma aplicação Java que se conecta a um banco de dados.

O IntelliJ IDEA é um ambiente de desenvolvimento (IDE) que oferece uma série de recursos úteis para programar com Java de forma mais organizada, produtiva e segura. Ele possui versões gratuitas (*Community*) e pagas (*Ultimate*), e para os propósitos desta prática, a versão gratuita é suficiente.

Se ainda não tiver o IntelliJ instalado, siga estes passos:

1. Acesse o site oficial: https://www.jetbrains.com/idea/download.
2. Baixe a versão *Community Edition***.**
3. Instale normalmente, mantendo as configurações padrão.

Depois da instalação, ao abrir o IntelliJ pela primeira vez, você verá a tela inicial com a opção **"*New Project*"**. Usaremos essa opção em breve.

O **MySQL** será nosso banco de dados neste projeto. Ele é amplamente usado no mercado e possui uma versão gratuita e de fácil instalação (Hernandez, 2021). Portanto, 1) Acesse o site: https://dev.mysql.com/downloads/installer; 2) Baixe a versão "MySQL *Installer Community*"; 3) Durante a instalação, anote a senha definida para o usuário root (administrador); e 4) Finalize a instalação e abra o **MySQL Workbench** (interface gráfica opcional).

Se você preferir utilizar o terminal, também poderá executar comandos diretamente por lá. A escolha entre interface gráfica ou terminal é sua, e ambas funcionam da mesma forma em termos de funcionalidade.

Estruturando o banco de dados– criação da tabela usuários

Com o MySQL devidamente instalado e funcionando em sua máquina, o próximo passo é criar o banco de dados e a tabela que armazenará os dados da aplicação Java (Hernandez, 2021; Oracle, 2020).

Abra o terminal ou o Workbench e conecte-se ao MySQL utilizando o usuário root e a senha definida durante a instalação. Em seguida, execute o seguinte comando: CREATE DATABASE sistema;. Esse comando criará um banco de dados chamado sistema, que será utilizado por nossa aplicação.

Com o banco criado, precisamos selecionar esse banco como o contexto atual e, em seguida, criar uma tabela chamada usuarios. Ela terá quatro campos: id, nome, email e senha. Se estiver usando o terminal, digite: USE sistema;. Agora, crie a tabela com o seguinte comando (Hernandez, 2021):

```
CREATE TABLE usuarios (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nome VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   senha VARCHAR(50) NOT NULL
);
```

Aqui, o campo id funciona como identificador único para cada usuário, sendo incrementado automaticamente a cada novo registro. O campo nome é obrigatório e pode conter até 100 caracteres. O email também é obrigatório e deve ser único, garantindo que dois usuários não possam utilizar o mesmo endereço. Já o campo senha armazena a senha do usuário em texto simples, com limite de 50 caracteres, uma solução provisória apenas para fins didáticos. A Figura 3 ilustra a execução dos comandos no MySQL Workbench.

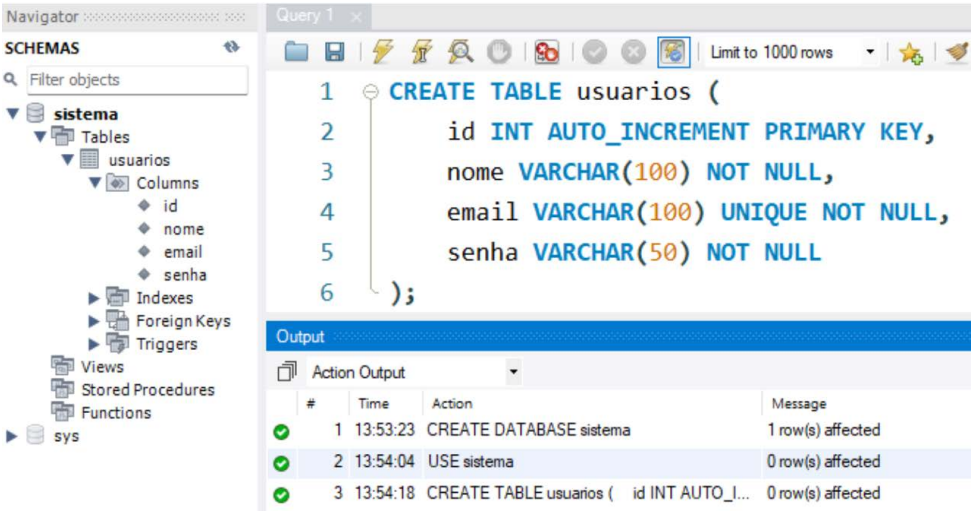


Figura 3 - Criação da Tabela usuarios no MySQL Fonte: o autor.

Descrição da Imagem: a imagem mostra a interface do MySQL Workbench com a criação de uma tabela chamada "usuarios" dentro do banco de dados "sistema". À esquerda, a estrutura do banco é exibida com a tabela usuarios e seus campos: id, nome, email e senha. No centro, há o código SQL utilizado para criar a tabela. Abaixo, a área de saída indica que os comandos CREATE DATABASE, USE e CREATE TABLE foram executados com sucesso, sem erros. Fim da descrição.

Para facilitar os testes da aplicação mais adiante, podemos já adicionar alguns registros à tabela:

```
INSERT INTO usuarios (nome, email, senha)
VALUES
  ('Ana Souza', 'ana@email.com', 'senha123'),
  ('Bruno Silva', 'bruno@email.com', 'bruno456');
```

Esses dados (ilustrados na Figura 4– estarão disponíveis para leitura assim que implementarmos a parte Java responsável pela consulta ao banco.

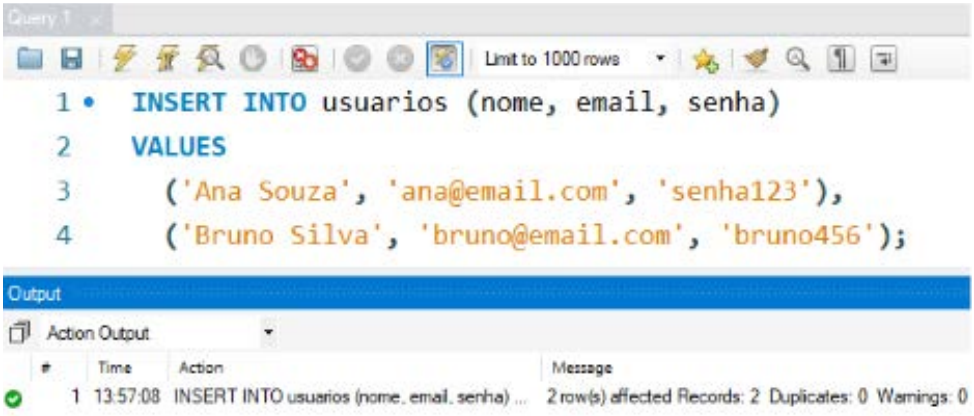


Figura 4 - Inserção de Dados na Tabela usuarios / Fonte: o autor.

Descrição da Imagem: a imagem mostra o comando SQL INSERT INTO sendo usado no MySQL Workbench para adicionar dois registros à tabela usuarios. Na parte inferior, o painel de saída confirma que dois registros foram adicionados com sucesso. Fim da descrição.

Com o banco sistema e a tabela "usuarios" prontos, estamos preparados para iniciar a criação da aplicação Java e configurá-la para se conectar ao banco.

Criando o projeto Java com Maven e Configurando o JDBC

Agora que o banco de dados está pronto, o próximo passo é criar o projeto Java que se conectará – ele. Utilizaremos o **IntelliJ IDEA** com **Maven**, uma combinação amplamente usada no mercado por sua praticidade, organização de dependências e escalabilidade.

Para criar o projeto, siga estes passos: 1) Abra o IntelliJ IDEA Community; 2) Na tela inicial, clique em ***New Project***; 3) No menu lateral esquerdo, selecione **Java**; 4) Em ***Build system***, escolha a opção **Maven**; 5) No campo **JDK**, selecione uma versão instalada (recomenda-se JDK 17 ou superior); 6) No campo ***Name***, digite o nome do seu projeto, como cadastro-usuarios; 7) Escolha uma pasta para salvar o projeto; e, por fim, 8) Clique em ***Create***.

Após alguns segundos, o IntelliJ criará a estrutura básica do projeto com Maven. No lado esquerdo, você verá o arquivo pom.xml (ver Figura 5), responsável por controlar as bibliotecas externas utilizadas na aplicação.

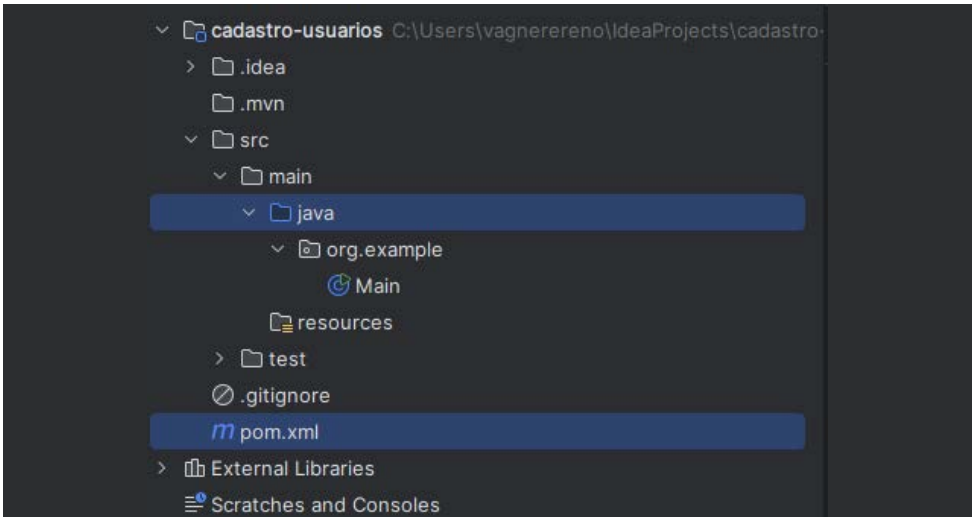


Figura 5 - Estrutura de Projeto Maven no IntelliJ IDEA / Fonte: o autor.

Descrição da Imagem: a imagem exibe a estrutura de pastas de um projeto Java chamado cadastro-usuarios, aberta no IntelliJ IDEA. Dentro da pasta src/main/java, há um pacote org.example contendo a classe Main.java. Também é visível o arquivo pom.xml, responsável pelas dependências e configurações do Maven. Fim da descrição.

O próximo passo é adicionar ao seu projeto a biblioteca que permite ao Java se comunicar com o MySQL. Para isso, edite o arquivo pom.xml e adicione o seguinte bloco dentro da tag <dependencies> (Oracle, 2020):

```
<dependencies>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
  </dependency>
</dependencies>
```

Após salvar o pom.xml, o IntelliJ deve automaticamente baixar o conector JDBC e adicioná-lo às *External Libraries* (lateral esquerda). Se não acontecer, clique com o botão direito no projeto e selecione ***Reload Maven Project***.

Construindo a aplicação Java com JDBC

Agora que temos o ambiente configurado, o banco de dados criado e o driver JDBC instalado no projeto, chegou o momento de começar a desenvolver a aplicação Java. Nessa etapa, vamos criar a estrutura inicial do projeto e implementar o código necessário para inserir dados na tabela usuarios.

Organizar bem seu projeto é essencial para facilitar a leitura, manutenção e evolução do código. Por isso, o primeiro passo é criar um pacote para agrupar as classes da aplicação. No IntelliJ, clique com o botão direito na pasta src/main/java, selecione New > Package, e crie um pacote com o nome: com.cadastro.usuarios. Dentro desse pacote, vamos criar três classes:

- **Usuario.java**: representa o modelo de dados.
- **Conexao.java**: gerencia a conexão com o banco.
- **Aplicacao.java**: classe principal da aplicação.

Vamos começar criando a classe **Usuario**, que representa os dados de um usuário no sistema. Essa classe será usada para **encapsular** as informações que serão armazenadas e recuperadas do banco de dados (Oracle, 2020):

```
package com.cadastro.usuarios;

public class Usuario {
    private int id;
    private String nome;
    private String email;
    private String senha;

    public Usuario(String nome, String email, String senha) {
        this.nome = nome;
        this.email = email;
        this.senha = senha;
    }

    public int getId() { return id; }
    public String getNome() { return nome; }
    public String getEmail() { return email; }
    public String getSenha() { return senha; }

    public void setId(int id) { this.id = id; }
    public void setNome(String nome) { this.nome = nome; }
    public void setEmail(String email) { this.email = email; }
    public void setSenha(String senha) { this.senha = senha; }
}
```

Note que o campo id não está presente no construtor, pois ele será gerado automaticamente pelo banco de dados. Por outro lado, é importante que todos os atributos tenham métodos *getters* e *setters* para que possamos usá-lo em operações futuras, como listagem e atualização (Oracle, 2020).

Agora, vamos popular a classe denominada **Conexao**, que será responsável por fornecer a conexão com o banco de dados MySQL. Isso ajuda a centralizar a lógica de conexão e evita duplicação de código (Oracle, 2020).

```
package com.cadastro.usuarios;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexao {
private static final String URL = "jdbc:mysql://localhost:3306/sistema";
   private static final String USUARIO = "root";
   private static final String SENHA = "sua_senha_aqui";

   public static Connection conectar() throws SQLException {
      return DriverManager.getConnection(URL, USUARIO, SENHA);
   }
}
```

Substitua o valor de SENHA pela senha que você configurou no MySQL. Esta classe é simples e retorna uma conexão sempre que for chamada.

Por fim, vamos popular a classe principal da aplicação. Inicialmente, ela será responsável por executar a inserção de um usuário no banco de dados. Essa inserção será feita por meio de um método chamado **inserirUsuario**:

```
package com.cadastro.usuarios;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class Aplicacao {

    public static void main(String[] args) {
        Usuario usuario = new Usuario("João da Silva", "joao@email.com", "123456");
        inserirUsuario(usuario);
    }

    public static void inserirUsuario(Usuario usuario) {
        String sql = "INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?)";

        try (Connection conn = Conexao.conectar();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, usuario.getNome());
            stmt.setString(2, usuario.getEmail());
            stmt.setString(3, usuario.getSenha());

            int linhasAfetadas = stmt.executeUpdate();
            System.out.println("Usuário inserido com sucesso! Linhas
afetadas: " + linhasAfetadas);

        } catch (SQLException e) {
            System.out.println("Erro ao inserir usuário: " + e.getMessage());
        }
    }
}
```

Organizar o código dessa forma torna a classe principal mais clara e permitirá que reutilizemos esse método futuramente, inclusive com diferentes entradas.

Implementando as operações CRUD completas

Agora que nossa aplicação já consegue inserir usuários no banco de dados, vamos complementar o sistema com as demais operações fundamentais do modelo CRUD: leitura (*Read*), atualização (*Update*) e exclusão (*Delete*).

Para manter o exemplo simples, continuaremos utilizando apenas a classe **Aplicacao**. Com isso, você poderá visualizar todas as operações reunidas em um único lugar e compreender o funcionamento prático de cada uma delas.

Vamos começar exibindo todos os usuários cadastrados na tabela usuarios:

```
public static void listarUsuarios() {
    String sql = "SELECT * FROM usuarios";

    try (Connection conn = Conexao.conectar();
        PreparedStatement stmt = conn.prepareStatement(sql);
        var rs = stmt.executeQuery()) {

        while (rs.next()) {
            int id = rs.getInt("id");
            String nome = rs.getString("nome");
            String email = rs.getString("email");
            String senha = rs.getString("senha");

            System.out.println(id + " | " + nome + " | " + email + " | " + senha);
        }

    } catch (SQLException e) {
        System.out.println("Erro ao listar usuários: " + e.getMessage());
    }
}
```

Em seguida, podemos modificar os dados de um usuário com base em seu ID:

```
public static void atualizarUsuario(int id, String novoNome, String novoEmail, String nova-
Senha) {
    String sql = "UPDATE usuarios SET nome = ?, email = ?, senha = ? WHERE id = ?";

    try (Connection conn = Conexao.conectar();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, novoNome);
        stmt.setString(2, novoEmail);
        stmt.setString(3, novaSenha);
        stmt.setInt(4, id);

        int linhasAfetadas = stmt.executeUpdate();
        if (linhasAfetadas > 0) {
            System.out.println("Usuário atualizado com sucesso.");
        } else {
            System.out.println("Nenhum usuário encontrado com o ID informado.");
        }

    } catch (SQLException e) {
        System.out.println("Erro ao atualizar usuário: " + e.getMessage());
    }
}
```

Por fim, a exclusão de usuários também pode ser feita de maneira simples:

```
public static void excluirUsuario(int id) {
    String sql = "DELETE FROM usuarios WHERE id = ?";

    try (Connection conn = Conexao.conectar();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, id);

        int linhasAfetadas = stmt.executeUpdate();
        if (linhasAfetadas > 0) {
            System.out.println("Usuário excluído com sucesso.");
        } else {
            System.out.println("Nenhum usuário encontrado com o ID informado.");
        }

    } catch (SQLException e) {
        System.out.println("Erro ao excluir usuário: " + e.getMessage());
    }
}
```

Para testar, basta chamar os métodos a partir do main da classe Aplicacao, por exemplo:

```
public class Aplicacao {

  public static void main(String[] args) {
    Usuario usuario = new Usuario("Vagner E.", "vagner@email.com", "0123564");
    listarUsuarios();
    inserirUsuario(usuario);
    atualizarUsuario(3, "João Atualizado");
    excluirUsuario(1);
    listarUsuarios();
  }
```

Esses exemplos fecham nosso ciclo de CRUD básico com JDBC (Hernandez, 2021). Mesmo sendo operações simples, elas são o alicerce para qualquer sistema que manipule dados. Ao compreender esse funcionamento, você, estudante, estará apto a evoluir para aplicações mais complexas no futuro.

EM FOCO

Deseja se aprofundar um pouco mais? Acompanhe a videoaula que apresenta esse conteúdo de forma prática e comentada.

NOVOS DESAFIOS

Ao longo deste tema, você percorreu os fundamentos que sustentam as aplicações web modernas. Entendeu como a arquitetura da internet se organiza, como requisições são processadas, como HTML e HTTP funcionam, e como o Java se conecta com bancos de dados relacionais por meio do JDBC. Cada conceito visto é parte de um ecossistema que viabiliza soluções digitais robustas e seguras.

Retomando a situação inicial do hospital que ainda utiliza fichas de papel: com os conhecimentos adquiridos, agora é possível vislumbrar uma solução viável. Ao integrar um formulário HTML a uma aplicação Java com JDBC, armazenando os dados em um banco como o MySQL, o tempo de atendimento é reduzido, os erros são minimizados e a informação flui entre os setores de forma controlada e acessível. Mais do que uma solução técnica, trata-se de um impacto direto na qualidade do serviço prestado.

Esse tipo de conhecimento é altamente valorizado no mercado de trabalho, especialmente em áreas que demandam a construção ou manutenção de sistemas internos, portais de atendimento, ERPs, CRMs e outras soluções corporativas. Dominar a base (isto é, saber como funcionam os protocolos, os padrões de integração e as estruturas de dados) permite que você vá além de copiar tutoriais. Você será capaz de diagnosticar, propor, adaptar e evoluir sistemas com mais autonomia e precisão.

Como profissional, você poderá não apenas implementar soluções, mas também argumentar tecnicamente com equipes e clientes, justificar escolhas tecnológicas e construir sistemas que se comuniquem de forma eficiente com o mundo real. Ao transformar um clique no botão "Enviar" em uma transação segura e útil, você também transformará a forma como organizações operam, e é exatamente aí que seu papel como especialista em Java para a Web começa a se destacar.

ATIVIDADES

1. O modelo cliente-servidor é amplamente adotado na comunicação entre aplicações web. Ele define papéis distintos para o cliente (por exemplo, o navegador) e para o servidor, que interagem por meio de protocolos específicos como o HTTP. Compreender como essa comunicação ocorre é essencial para o desenvolvimento *backend*, já que o servidor precisa saber interpretar e responder corretamente às requisições recebidas.

   Com base nos conceitos relacionados ao modelo cliente-servidor e à comunicação na Web, avalie as afirmativas a seguir:

   I - Em aplicações Web, é comum que o cliente envie requisições HTTP para o servidor, que responde com conteúdos como páginas HTML ou dados estruturados.
   II - O navegador, ao acessar um endereço web, é responsável por interpretar o HTML retornado pelo servidor e exibir o conteúdo ao usuário.
   III - O servidor pode utilizar ferramentas como o Tomcat para receber requisições e executar o código Java associado à lógica da aplicação.
   IV - Cabe ao cliente executar o código Java que interpreta a requisição e constrói a resposta a ser enviada de volta ao servidor.

   É correto o que se afirma em:

   a) I, apenas.
   b) II e IV, apenas.
   c) III e IV, apenas.
   d) I, II e III, apenas.
   e) I, II, III e IV.

2. Sistemas web utilizam a linguagem SQL para interagir com bancos de dados relacionais. As operações básicas de inserção, consulta, atualização e exclusão (conhecidas como CRUD) são executadas por meio de comandos específicos da linguagem. A correta utilização desses comandos permite que aplicações armazenem e gerenciem dados de forma eficiente, garantindo consistência e integridade nas transações.

   Considere o seguinte cenário: uma aplicação precisa atualizar o endereço de e-mail de um usuário cujo identificador (id) é 7. Para isso, deve ser utilizado um comando SQL apropriado. Qual das alternativas a seguir representa corretamente essa instrução?

   a) UPDATE usuarios SET email = 'novo@email.com'.
   b) MODIFY usuarios SET email = 'novo@email.com' WHERE id = 7.
   c) UPDATE usuarios SET email = 'novo@email.com' WHERE id = 7.
   d) ALTER TABLE usuarios CHANGE email TO 'novo@email.com' WHERE id = 7.
   e) SET usuarios.email = 'novo@email.com' WHERE id = 7.

3. O JDBC é a API padrão do Java utilizada para estabelecer conexões com bancos de dados relacionais, enviar comandos SQL e manipular os resultados. O processo de comunicação via JDBC segue uma sequência de etapas: carregamento do driver, criação da conexão, preparação da instrução, execução da consulta e, por fim, leitura dos resultados. Compreender essa estrutura é fundamental para o desenvolvimento de aplicações Java seguras e eficientes.

   Durante o desenvolvimento de uma aplicação Java conectada a um banco MySQL via JDBC, um estudante organizou a lógica de conexão da seguinte forma:

   1 - Estabelecimento da conexão com DriverManager.getConnection(...).
   2 - Execução de um comando SQL com Statement.executeQuery(...).
   3 - Leitura dos dados com ResultSet.next().

   Considerando as boas práticas e a sequência adequada do JDBC, qual dos elementos abaixo ainda **precisa ser adicionado** ao processo descrito?

   a) Criação de uma URL de conexão no formato JDBC e carregamento explícito do driver.
   b) Declaração da variável main() como throws SQLException.
   c) Uso obrigatório de comandos INSERT no lugar de SELECT.
   d) Implementação de uma interface gráfica para entrada dos dados.
   e) Criação manual de uma chave primária na tabela acessada.

REFERÊNCIAS

HERNANDEZ, M. J. **Database design for mere mortals**: A hands-on guide to relational database design. 4. ed. 25th Anniversary Edition. Boston: Addison-Wesley, 2021.

KUROSE, J. F.; ROSS, K. W. **Computer networking**: A top-down approach. 8. ed. Boston: Pearson, 2020.

ORACLE. **Oracle database JDBC developer's guide, 19c. 1**. ed. Redwood Shores, CA: Oracle, 2023. Disponível em: https://docs.oracle.com/en/database/oracle/oracle-database/19/jjdbc/index.html. Acesso em: 1 jul. 2025.

WOLF, J. **HTML & CSS**: The comprehensive guide to excelling in HTML5 and CSS3 for responsive web design, dynamic content, and modern layouts. 1. ed. Bonn: Rheinwerk Computing, 2023.

GABARITO

1. Alternativa D.

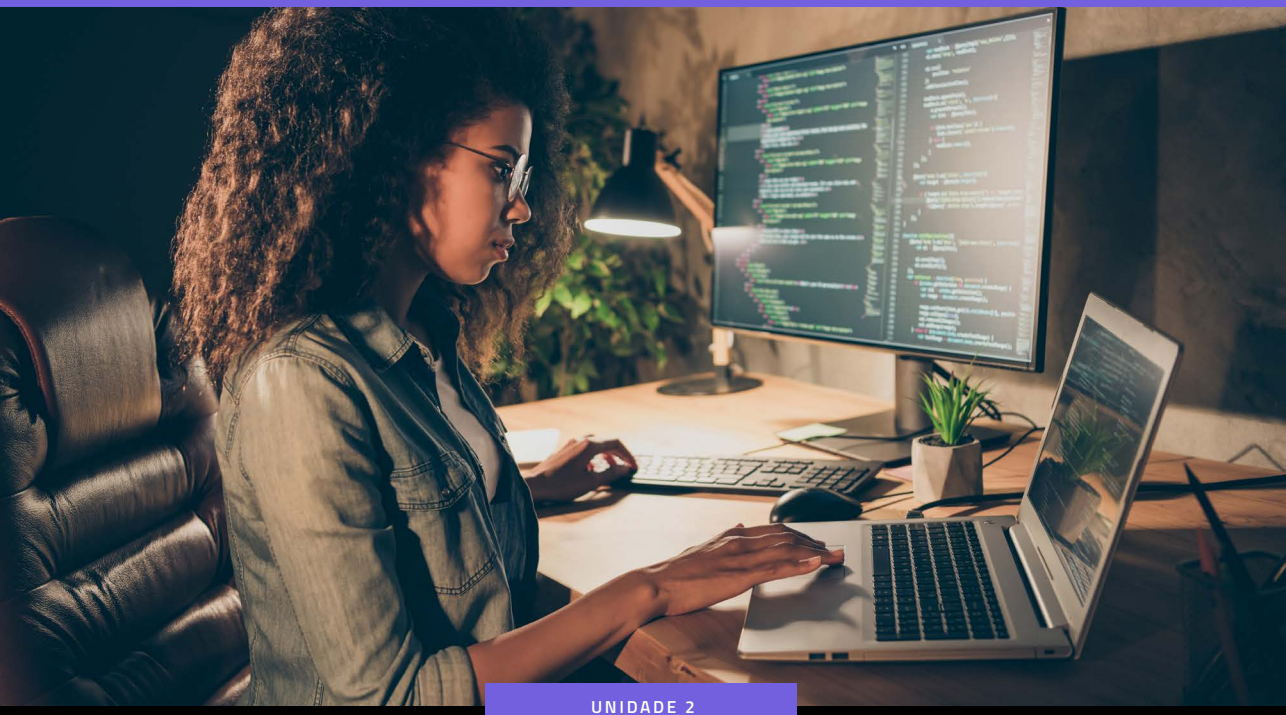
   "Em aplicações Web, é comum que o cliente envie requisições HTTP para o servidor, que responde com conteúdos como páginas HTML ou dados estruturados."
   I. Correta. Essa é a base da comunicação na Web moderna.
   "O navegador, ao acessar um endereço web, é responsável por interpretar o HTML retornado pelo servidor e exibir o conteúdo ao usuário."
   II.Correta. O navegador não cria o HTML, mas o interpreta.
   "O servidor pode utilizar ferramentas como o Tomcat para receber requisições e executar o código Java associado à lógica da aplicação."
   III.Correta. O Tomcat é comumente usado como container de servlets.
   "Cabe ao cliente executar o código Java que interpreta a requisição e constrói a resposta a ser enviada de volta ao servidor."
   IV.Incorreta. Essa tarefa é exclusiva do servidor; o cliente não executa código Java do backend.

2. Alternativa C.

   "UPDATE usuarios SET email = 'novo@email.com';"
   A. Incorreta. A ausência de cláusula WHERE faz com que o e-mail de todos os usuários seja alterado, o que pode comprometer a integridade dos dados.
   "MODIFY usuarios SET email = 'novo@email.com' WHERE id = 7;"
   B. Incorreta. MODIFY não é um comando válido em SQL para atualizar dados em tabelas.
   "UPDATE usuarios SET email = 'novo@email.com' WHERE id = 7;"
   C. Correta. Essa é a sintaxe adequada do comando UPDATE, utilizando a cláusula WHERE para filtrar o registro específico.
   "ALTER TABLE usuarios CHANGE email TO 'novo@email.com' WHERE id = 7;"
   D. Incorreta. ALTER TABLE é utilizado para modificar a estrutura da tabela, não os dados nela contidos.
   "SET usuarios.email = 'novo@email.com' WHERE id = 7;"
   E. Incorreta. A instrução SET isoladamente não é válida fora de um contexto de UPDATE neste tipo de operação.

3. Alternativa A.

"Criação de uma URL de conexão no formato JDBC e carregamento explícito do driver."
A. Correta. Antes de usar DriverManager.getConnection(...), é necessário que a URL esteja corretamente formatada (como jdbc:mysql://localhost:3306/sistema) e o driver esteja carregado (o que geralmente ocorre automaticamente via Maven, mas pode exigir registro manual em versões antigas).
"Declaração da variável main() como throws SQLException."
B. Incorreta. Embora isso evite tratamento local de exceções, não é um passo obrigatório para a funcionalidade do JDBC, apenas uma opção de design.
"Uso obrigatório de comandos INSERT no lugar de SELECT."
C. Incorreta. Comandos SELECT são perfeitamente válidos no uso de executeQuery() para consultas.
"Implementação de uma interface gráfica para entrada dos dados."
D. Incorreta. A interface gráfica é opcional e não interfere na lógica de conexão JDBC.
"Criação manual de uma chave primária na tabela acessada."
E. Incorreta. A definição da estrutura do banco (incluindo chaves primárias) é importante, mas não faz parte da sequência de conexão via JDBC em si.



UNIDADE 2

JAVA SERVLET E JAVASERVER PAGES (JSP)

MINHAS METAS

- Compreender o ciclo de vida dos Servlets Java.
- Criar aplicações web com Servlets e JSPs.
- Elaborar conteúdo dinâmico com páginas JSP.
- Gerenciar sessões e cookies em aplicações web.

INICIE SUA JORNADA

Imagine que você foi contratado para modernizar um sistema interno de cadastro de funcionários de uma empresa. O sistema anterior era feito em Java, mas todo o processo era manual: os dados eram inseridos diretamente no banco por scripts, sem nenhuma interface amigável. Agora, a empresa deseja algo novo — páginas acessadas por navegador, onde o usuário possa preencher um formulário, clicar em "Salvar" e ver os dados sendo registrados automaticamente no banco. O objetivo é tornar o sistema mais intuitivo e seguro para os colaboradores que lidam com os cadastros diariamente.

Parece simples? Nem tanto. Para que isso aconteça, a aplicação precisa entender as requisições do navegador, processar os dados no servidor e responder adequadamente. Mais do que isso: é necessário saber controlar as rotas, tratar formulários, exibir feedback ao usuário e manter alguma forma de controle da sessão enquanto ele navega. Tudo isso precisa acontecer de forma eficiente, sem comprometer o desempenho nem a segurança da aplicação. E é exatamente aqui que entram os Servlets e as JSPs: duas das tecnologias mais importantes quando falamos de desenvolvimento web com Java.

Com esse desafio profissional, você começa a perceber que saber apenas escrever classes Java e manipular o banco de dados com JDBC não é suficiente. Agora você precisa entender o que acontece "entre o navegador e o banco", como os dados circulam, como o servidor reage, e principalmente, como gerar páginas dinâmicas que se adaptam ao que o usuário faz. É nesse espaço que entram as habilidades que vamos explorar a partir de agora, e que fazem toda a diferença no desenvolvimento de aplicações web robustas.

Ao aprender a lidar com o ciclo de vida dos Servlets, gerar conteúdo dinâmico com JSP e controlar sessões de forma eficaz, você não apenas amplia sua capacidade técnica, mas se aproxima das exigências reais do mercado. Essas competências são essenciais para quem deseja criar sistemas web completos e bem estruturados. E o melhor: tudo isso servirá como base para avançar com segurança rumo a soluções ainda mais sofisticadas e modernas.

PLAY NO CONHECIMENTO

Curioso para saber como Servlets se conectam ao mundo das APIs e microserviços? Ouça nosso podcast e descubra como esse conhecimento pode abrir portas para o desenvolvimento moderno em Java Web!

DESENVOLVA SEU POTENCIAL

SERVLETS NO DESENVOLVIMENTO WEB JAVA

Você já se perguntou como uma aplicação Java pode "entender" o que o usuário faz em uma página web? Por exemplo, quando alguém preenche um formulário e clica em "Enviar", o que acontece do lado do servidor? É aí que entram os **Servlets** (Juneau, 2020).

Um **Servlet** é uma classe Java que roda dentro de um servidor de aplicações e atua como o cérebro por trás das interações web. Quando um navegador faz uma requisição HTTP (como acessar uma página ou enviar um formulário), o Servlet entra em ação para decidir o que fazer com essa informação: processar dados, consultar o banco, redirecionar o usuário ou gerar uma resposta personalizada (Juneau, 2020).

Para facilitar, pense nos Servlets como os **"porteiros"** de uma aplicação web: eles recebem os pedidos de quem chega (o navegador), analisam o que está sendo pedido e respondem adequadamente, seja com uma página, uma mensagem ou uma ação no sistema.

Esses componentes fazem parte da especificação **Java EE** (atualmente Jakarta EE) e são essenciais para criar aplicações web que vão além do HTML estático. Com eles, conseguimos adicionar lógica de programação entre o que o usuário vê na tela e o que está por trás do sistema.

Ao longo deste conteúdo, veremos como esses Servlets funcionam na prática, como são estruturados e de que maneira se conectam com outras tecnologias web, como o JSP. Mas por enquanto, guarde a ideia central: Servlets são o ponto de entrada para processar requisições e gerar respostas dinâmicas em uma aplicação Java web.

Ciclo de vida de um Servlet

Agora que você já compreendeu o papel dos Servlets no desenvolvimento de aplicações Java web, é hora de entender como eles funcionam por dentro, no ambiente do servidor. Esse funcionamento segue um ciclo bem definido, conhecido como **ciclo de vida do Servlet** (Juneau, 2020).

Esse ciclo começa quando o servidor carrega a aplicação pela primeira vez ou quando um Servlet é requisitado pela primeira vez. A partir daí, a instância do Servlet é criada e mantida em memória, pronta para processar requisições sempre que necessário. Para ilustrar esse processo, observe a figura a seguir:

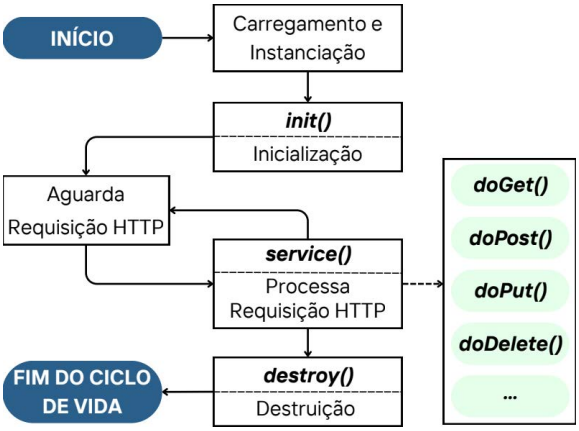


Figura 1 – Ciclo de Vida de um Servlet. Fonte: o autor.

Descrição da Imagem: a imagem mostra um fluxograma com o ciclo de vida de um servlet. Inicia em "INÍCIO", passa pelas etapas de carregamento, inicialização (init()), espera por requisições HTTP, processa as requisições HTTP (service()), e finalização (destroy()). O método service() delega as requisições para doGet(), doPost(), doPut(), doDelete() e outros. Fim da descrição.

O ciclo de vida de um Servlet é composto pelas seguintes etapas:

- **Carregamento e Instanciação:** o servidor identifica a necessidade de um Servlet e carrega sua classe. Isso pode ocorrer logo na inicialização da aplicação ou sob demanda, quando a primeira requisição é feita.
- **Inicialização – init()**: após instanciar o Servlet, o servidor chama o método init(), que serve para inicializar configurações específicas. Esse método é executado apenas uma vez durante todo o ciclo de vida.
- **Processamento de Requisições – service()**: com o Servlet pronto, ele entra em um estado de espera por requisições HTTP. Quando uma requisição chega, o método service() é acionado para tratá-la. Esse método identifica o tipo da requisição (GET, POST etc.) e encaminha para o método correspondente: doGet(), doPost(), doPut(), entre outros.
- **Destruição – destroy()**: quando o servidor precisa encerrar a aplicação ou liberar recursos, ele chama o método destroy(), finalizando o ciclo de vida do Servlet.

Esse ciclo permite que o Servlet trate múltiplas requisições ao longo do tempo sem ser recriado a cada acesso (Murach; Urban, 2014). Isso torna sua utilização mais eficiente e controlada. Compreender esse ciclo é essencial para estruturar corretamente a lógica de uma aplicação web e evitar comportamentos inesperados, como erros de concorrência ou perda de dados.

Funcionamento de uma aplicação Servlet

Agora que você já compreendeu o que são Servlets e como eles se comportam durante seu ciclo de vida, é hora de entender o caminho completo que uma requisição percorre ao sair do navegador até ser processada pelo servidor.

Tudo começa quando o usuário interage com o navegador, por exemplo, ao digitar um endereço ou clicar em um botão que leva a uma página. Suponha que alguém acesse a URL “http://localhost:8080/meuapp/ola”. Essa requisição chega ao servidor web (como o Apache Tomcat), que está escutando naquela porta (8080) e gerenciando a aplicação chamada meuapp.

EU INDICO

O Apache Tomcat é um servidor *open source* usado para executar aplicações Java baseadas em Servlets e JSP. No site oficial (tomcat.apache.org), você encontra downloads de várias versões, documentação completa, tutoriais e guias de configuração. Ideal para aprofundar seu entendimento técnico sobre como o Tomcat atua nesse processo.

O servidor, então, analisa o caminho /ola e verifica qual Servlet está configurado para responder a essa rota. Caso encontre um Servlet mapeado para esse endereço, ele cria (ou reutiliza) uma instância dessa classe e chama o método adequado, geralmente doGet() ou doPost(), dependendo do tipo da requisição. Esse método é responsável por processar os dados, executar a lógica necessária e gerar uma resposta, que geralmente é um conteúdo HTML ou outro tipo de dado (como JSON, XML etc.) (Caelum, 2021).

Essa resposta é então enviada de volta ao navegador, que a exibe na tela. O usuário, muitas vezes, nem imagina que por trás daquele clique simples houve todo um ciclo de processamento, com diversas etapas envolvidas entre o cliente e o servidor.

Se você pensar bem, a comunicação com Servlets segue uma lógica bem estruturada: o navegador envia uma requisição, o Servlet interpreta e responde (Caelum, 2021).

ENTENDENDO O PROCESSAMENTO DE REQUISIÇÕES

Você já sabe que aplicações web funcionam com base na troca de informações entre navegador e servidor. Essa troca acontece por meio do **protocolo HTTP** — uma espécie de linguagem combinada que define como as mensagens são enviadas e recebidas na web (Caelum, 2021).

Entre os diversos métodos definidos por esse protocolo, os mais comuns e importantes para aplicações Java com Servlets são o **GET** e o **POST**. Ambos servem para solicitar uma resposta do servidor, mas têm propósitos e comportamentos diferentes.

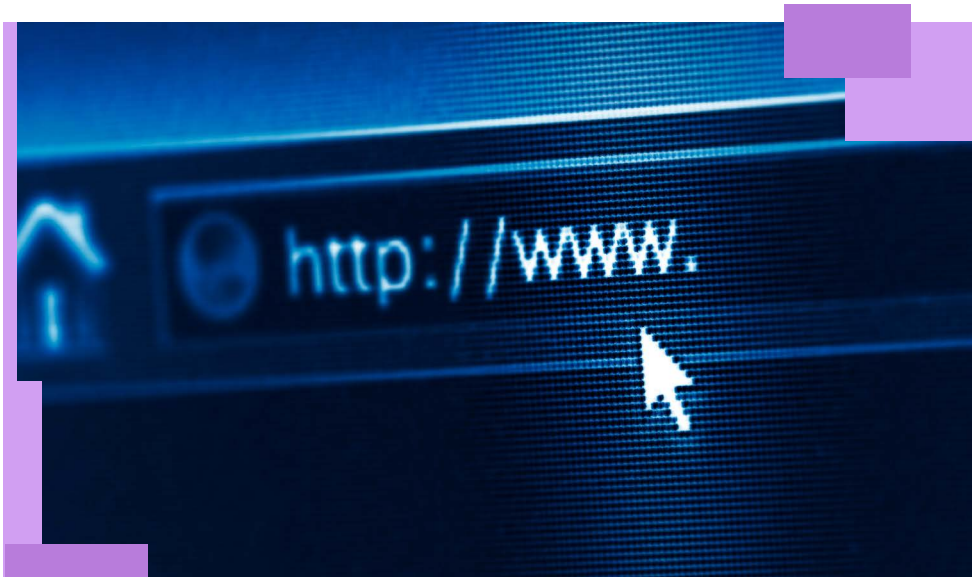
O método **GET** é geralmente usado para recuperar informações. Quando você acessa uma página, faz uma busca ou clica em um link, o navegador geralmente envia uma requisição GET. Os dados são transmitidos pela URL, o que torna esse método mais simples e rápido, porém menos seguro para o envio de dados sensíveis.

Já o método **POST** é utilizado quando se deseja enviar informações ao servidor, como ao preencher e submeter um formulário. Nesse caso, os dados são incluídos no corpo da requisição, o que permite o envio de grandes volumes e garante mais segurança, já que não ficam visíveis na barra de endereços.

No universo dos Servlets, o Java oferece duas formas principais de lidar com esses métodos: os métodos **doGet()** e **doPost()** (Caelum, 2021). Sempre que o servidor recebe uma requisição, ele verifica qual foi o método HTTP usado e chama a função correspondente dentro do Servlet.

Mapeamento de URLs e parâmetros em Servlets

Imagine que um usuário acessa o endereço www.sistema.com/usuarios. Como o servidor sabe qual parte do seu código Java deve responder a essa requisição? A resposta está no mapeamento de URLs (Juneau, 2020).



Em uma aplicação Java Web com Servlets, você pode indicar, por meio de anotações ou arquivos de configuração, quais URLs devem ser associadas a quais Servlets. Isso significa que, ao digitar um determinado endereço, o servidor saberá qual classe deve ser executada para gerar a resposta.

A forma mais comum de fazer isso atualmente é com a anotação @WebServlet (Juneau, 2020). Veja um exemplo simplificado:

```
@WebServlet("/usuarios")
public class UsuarioServlet extends HttpServlet {
    // código do Servlet aqui
}
```

Nesse caso, sempre que o navegador acessar /usuarios, o servidor executará o código presente em UsuarioServlet.

Mas, além de identificar qual classe deve responder, o Servlet também precisa lidar com os parâmetros enviados na requisição, especialmente em métodos GET e POST. Esses parâmetros podem ser dados digitados pelo usuário em formulários ou valores passados pela URL (Juneau, 2020).

Por exemplo, se o navegador acessar: /usuarios?nome=Ana&idade=25, o Servlet pode capturar esses dados com:

```
String nome = request.getParameter("nome");
String idade = request.getParameter("idade");
```

Esses métodos permitem extrair valores individuais de campos preenchidos no navegador, facilitando a construção de páginas dinâmicas baseadas no que o usuário deseja.

É importante mencionar que, para que um Servlet entre em ação, ele precisa estar corretamente mapeado a uma URL. Esse mapeamento é o que permite ao servidor saber qual classe Java deve responder a determinada rota acessada pelo navegador.

INTRODUÇÃO AO JAVASERVER PAGES (JSP)

Quando você acessa um site moderno, é natural esperar que a página mude de acordo com o seu comportamento. Seja exibindo seu nome, resultados de uma busca ou notificações personalizadas, tudo isso exige **conteúdo dinâmico**.

Na programação Java para web, uma das primeiras tecnologias desenvolvidas para esse propósito foi o *JavaServer Pages* (JSP) (Juneau, 2020). Com ela, tornou-se possível criar páginas que não apenas exibem conteúdo estático em HTML, mas que também podem incluir código Java embutido, responsável por gerar dinamicamente o que o usuário vê na tela.

O JSP foi projetado para trabalhar em conjunto com os Servlets: enquanto os Servlets cuidam da **lógica** de **controle**, o JSP fica responsável pela **apresentação** da informação, ou seja, o que o usuário vê no navegador.

HTML Estático versus JSP

Já vimos que o JSP surgiu como uma solução para gerar conteúdo **dinâmico** nas páginas web, mas, afinal, o que isso significa na prática? Para entender melhor, precisamos distinguir dois tipos de conteúdo: o estático e o dinâmico.

O conteúdo estático, como o nome indica, não muda. Se escrevermos um arquivo HTML simples com uma mensagem de boas-vindas, todos os usuários verão a mesma mensagem, sempre. É como imprimir um panfleto: o conteúdo é fixo, impresso uma única vez e distribuído para todos da mesma forma.

Conteúdo estático, como o nome indica, não muda

```
<h1>Bem-vindo, usuário!</h1>
```

Veja um exemplo de conteúdo estático acima (Wolf, 2023). Tal conteúdo representa uma página HTML tradicional, que pode ser acessada por qualquer pessoa e retornará sempre o mesmo resultado, independente de quem está acessando.

Já o conteúdo dinâmico é gerado no momento da requisição, com base nas informações do usuário, do sistema ou do contexto (Wolf, 2023). Imagine agora que queremos exibir o nome da pessoa logada na plataforma:

```
<h1>Bem-vindo, <%= nomeUsuario %>!</h1>
```

Nesse caso, o servidor irá processar a variável "nomeUsuario" e gerar uma página personalizada antes de enviá-la ao navegador. É como se cada pessoa recebesse um panfleto exclusivo, com informações impressas sob medida.

Essa é justamente a proposta do JSP: permitir que páginas HTML ganhem dinamismo ao incorporar lógica de programação Java. Isso torna possível responder de forma inteligente às ações do usuário, apresentar dados vindos do banco de dados e adaptar a interface de acordo com o fluxo da aplicação.

Saber quando utilizar conteúdo estático ou dinâmico é uma habilidade essencial para todo desenvolvedor web, pois impacta diretamente na performance, na organização do sistema e na experiência do usuário.

Sintaxe básica do JSP e expressões

Agora que você já entendeu o papel do JSP para gerar conteúdo dinâmico, é hora de conhecer a estrutura básica de uma página JSP e como ela permite misturar HTML com trechos de código Java.

Uma página JSP é, essencialmente, um arquivo com extensão .jsp que contém HTML como qualquer outra página da web, mas com a vantagem de poder incluir **instruções em Java** que serão processadas no servidor antes de enviar o resultado ao navegador (Murach; Urban, 2014).

Existem quatro formas principais de incorporar Java em uma página JSP:

- **Expressões (<%= ... %>)**

Usadas para exibir valores diretamente no HTML gerado. Por exemplo:
Seu nome é: <%= nomeUsuario %>
Aqui, o valor da variável nomeUsuario será impresso no local onde a expressão aparece.

- **Scriptlets (<% ... %>)**

Permitem escrever blocos de código Java dentro da página. Esse recurso deve ser usado com moderação, já que o ideal é manter a lógica no Servlet ou em outras camadas. Veja um exemplo:

```
<%
int idade = 25;
%>
<p>Idade: <%= idade %></p>
```

Apesar de funcionar, essa prática está cada vez mais em desuso por dificultar a manutenção do código.

- **Diretivas (<%@ ... %>)**

Servem para configurar aspectos da página, como importar bibliotecas Java. Um exemplo comum é:

```
<%@ page contentType="text/html;charset=UTF-8" %>
```

Essa diretiva define o tipo de conteúdo da página e o conjunto de caracteres, garantindo que ela seja exibida corretamente em diferentes navegadores.

- ***Expression Language* (EL) ($[...])**

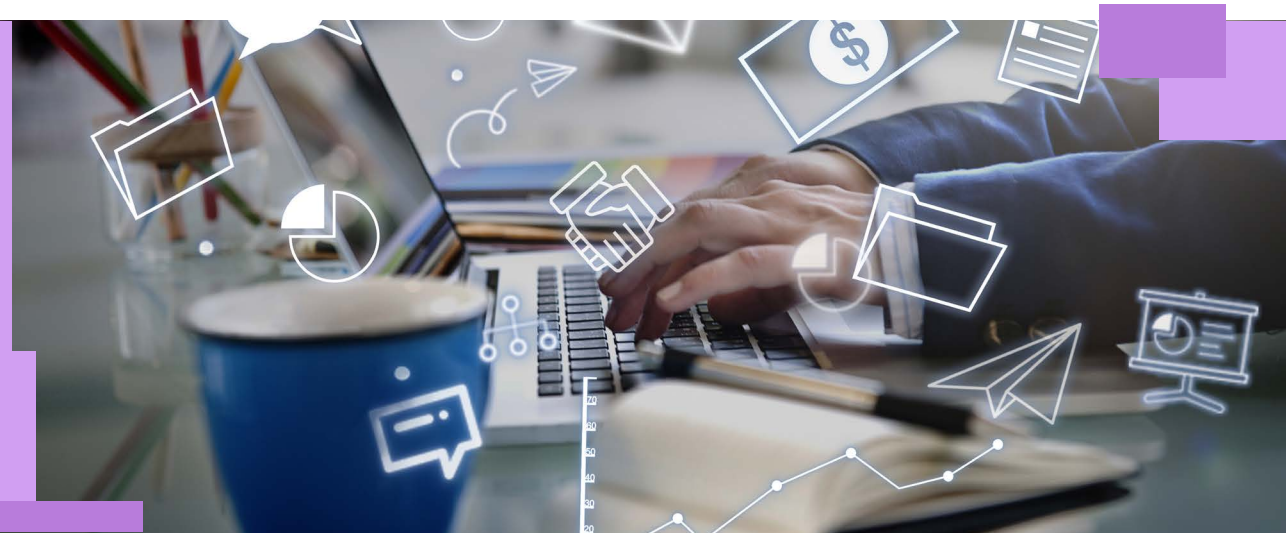
É a abordagem moderna e recomendada para exibir valores em páginas JSP. Sua sintaxe é mais limpa, evita erros com valores nulos e mantém a separação entre lógica e apresentação. Por exemplo:
Seu nome é: ${nomeUsuario}

Com EL, o JSP acessa atributos definidos no request, session ou application, facilitando a manutenção do código e aproximando o JSP de boas práticas como o padrão MVC.

Esses elementos formam a base da programação com JSP. Embora seja possível escrever lógica diretamente na página, uma boa prática é utilizar o JSP apenas para exibir informações, deixando o processamento de dados para os Servlets. Isso mantém o código mais limpo, organizado e fácil de evoluir.

Exemplo de geração de página dinâmica

Depois de entender a sintaxe básica do JSP, que tal ver um exemplo prático funcionando? Vamos criar uma página que receba um nome vindo de um formulário HTML e exiba uma saudação personalizada usando JSP (Murach; Urban, 2014). O objetivo aqui é mostrar, de forma simples, como o conteúdo dinâmico é gerado com base em dados enviados pelo usuário.



Imagine que temos o seguinte formulário HTML (ele pode estar em uma página estática ou em um JSP separado):

```
<form action="saudacao.jsp" method="get">
<label for="nome">Digite seu nome:</label>
<input type="text" id="nome" name="nome">
<button type="submit">Enviar</button>
</form>
```

Esse formulário envia os dados para a página **saudacao.jsp**, que será responsável por receber o parâmetro nome e exibir uma resposta dinâmica:

```
<%@ page contentType="text/html;charset=UTF-8" %>

<!DOCTYPE html>
<html>
<head>
    <title>Saudação</title>
</head>
<body>
    <h1>Olá, <%= request.getParameter("nome") %>! Seja bem-vindo(a).</h1>
</body>
</html>
```

Observe o funcionamento: ao preencher o campo e clicar em "Enviar", o navegador envia o valor digitado como parâmetro da URL. O JSP recupera esse valor com request.getParameter("nome") e o insere diretamente no HTML que será exibido para o usuário.

Esse é um dos usos mais básicos (e poderosos) do JSP: tornar o conteúdo das páginas sensível aos dados que chegam de formulários, URLs ou outras fontes (Murach e Urban, 2014). A partir daqui, podemos construir experiências mais interessantes, como exibir resultados de busca, mostrar mensagens personalizadas ou apresentar dados recuperados de um banco.

INTEGRAÇÃO ENTRE SERVLETS E JSP

À medida que as aplicações web crescem em complexidade, separar bem as responsabilidades de cada parte do sistema torna-se essencial. Quando usamos apenas Servlets, toda a lógica de negócio e a apresentação acabam ficando misturadas em uma única classe Java, o que dificulta a manutenção e a reutilização.

Por outro lado, o JSP sozinho não é ideal para executar regras de negócio ou acessar o banco de dados (Juneau, 2020).

Por isso, a boa prática é dividir os papéis da seguinte forma:

- O **Servlet** atua como controlador: ele recebe as requisições, processa os dados, acessa o banco quando necessário e decide qual página será exibida.
- O **JSP** é responsável apenas por exibir os dados: ele apresenta a resposta ao usuário de forma visual, usando HTML combinado com pequenos trechos de código Java, se necessário.

Essa separação segue o padrão arquitetural **MVC (*Model-View-Controller*)**, muito utilizado em aplicações web. Embora o padrão MVC seja abordado em mais profundidade mais adiante, aqui já começamos a aplicá-lo de maneira prática. Veja um exemplo simplificado na Figura 2, que ilustra esse fluxo:

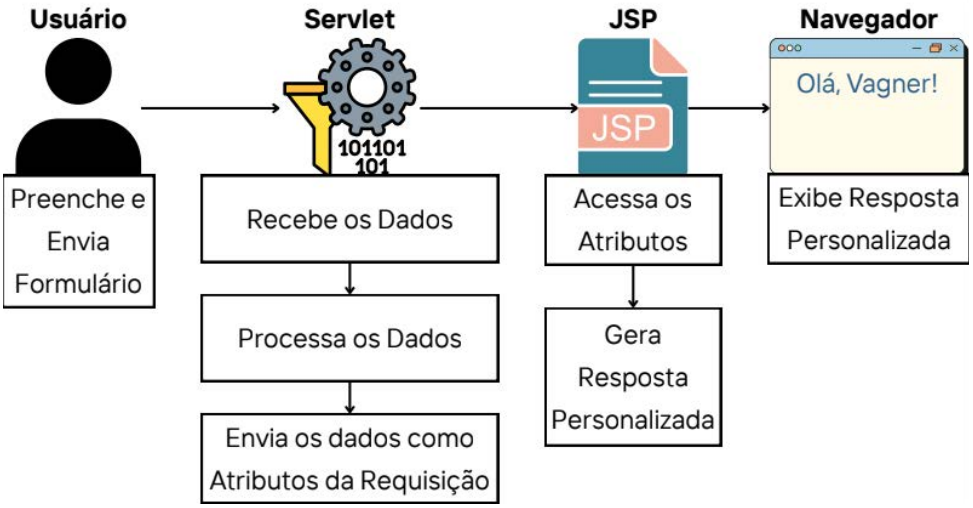


Figura 2 – Integração entre Servlet e JSP no padrão MVC. / Fonte: o autor.

Descrição da Imagem: a figura mostra um fluxo entre quatro elementos: o usuário, o Servlet, o JSP e o Navegador. O usuário envia uma requisição HTTP por meio de um formulário. O Servlet recebe e processa os dados, e então envia esses dados como atributos da requisição. Em seguida, o JSP acessa os atributos e gera uma resposta personalizada que é exibida no Navegador. O modelo segue o padrão MVC: o Servlet atua como controlador e o JSP como a camada de visualização. Fim da descrição.

No exemplo da imagem, o usuário envia um formulário com seu nome. O Servlet atua como **controlador**: ele recebe os dados do formulário, processa a lógica necessária e envia os dados como atributos da requisição. Logo após, o controle é encaminhado internamente para o **JSP**, que acessa esses atributos e gera uma resposta personalizada, como uma saudação ou confirmação, que será exibida ao usuário no navegador (Juneau, 2020).

Esse tipo de organização facilita muito a manutenção do código e permite que *designers* trabalhem na interface (JSP) enquanto desenvolvedores cuidam da lógica (Servlets).

Encaminhamento de dados com RequestDispatcher

A figura anterior ilustrou bem o papel dos Servlets e JSPs em uma aplicação web baseada no padrão MVC. O Servlet **recebe** a requisição, **processa** os dados e **transfere** as informações para a camada de visualização (o JSP), que monta a **resposta** exibida ao usuário (Caelum, 2021).

VOCÊ SABE RESPONDER?

Como essa transferência acontece, na prática?

É aqui que entra o RequestDispatcher. Ele é o componente responsável por encaminhar o fluxo de execução, ainda dentro do servidor, para outro recurso, como uma página JSP. E o mais importante: ele permite que o Servlet anexe dados à requisição, que poderão ser acessados pelo JSP.

Na prática, o uso segue dois passos principais (Juneau, 2020):

1. **Armazenar dados na requisição**: o Servlet insere os dados com o método setAttribute, por exemplo:
   request.setAttribute("nomeUsuario", "João");

2. **Encaminhar a requisição para o JSP**: com os dados já disponíveis na requisição, o Servlet repassa o controle:
   RequestDispatcher rd =
   request.getRequestDispatcher("boasvindas.jsp");
   rd.forward(request, response);

Do lado do JSP, podemos recuperar o dado com:

```
<p>Olá, <%= request.getAttribute("nomeUsuario") %>!</p>
```

Esse encaminhamento ocorre sem redirecionamento de URL, ou seja, o navegador continua acreditando que está na mesma rota inicial. Isso torna a resposta mais rápida e evita a exposição de rotas internas (Juneau, 2020).

Além de separar responsabilidades, esse fluxo facilita a manutenção da aplicação e prepara o terreno para adoção de frameworks mais robustos no futuro, como o *JavaServer Faces* (JSF), que automatiza parte desse processo.

MINIAPLICAÇÃO INTEGRADA: SERVLET & JSP

Chegou o momento de conectar os pontos. Vamos construir uma aplicação simples que recebe o nome do usuário por meio de um formulário HTML, processa essa informação com um Servlet e apresenta uma saudação personalizada em uma página JSP (Caelum, 2021). Essa é a base de inúmeras aplicações web reais: **entrada** de dados, **processamento** e **resposta**.

Configuração do ambiente

Para o exemplo desta miniaplicação integrada com Servlet e JSP, utilizaremos o IntelliJ IDEA Ultimate, pois essa versão da IDE oferece suporte nativo a desenvolvimento web, incluindo integração direta com servidores de aplicação como o Apache Tomcat. Isso torna o processo de execução mais simples e direto, especialmente para visualizar a aplicação rodando no navegador sem a necessidade de configurações manuais complexas.

APROFUNDANDO

A partir do Tomcat 10, houve uma mudança importante: a especificação javax.servlet passou a se chamar jakarta.servlet. Isso significa que projetos configurados com a nova API Jakarta não funcionam corretamente com versões antigas do Tomcat (como 9.x), e o contrário também é verdadeiro (Eclipse Foundation, 2022). Por isso, recomendamos utilizar **Tomcat 10.1 ou superior** para aplicações que usam jakarta.servlet, como é o caso deste projeto. Ao configurar o servidor no IntelliJ, certifique-se de que o diretório do Tomcat esteja correto e que a versão esteja sendo reconhecida pela IDE.

A seguir, mostramos o passo a passo para criar um projeto compatível e executar a aplicação desenvolvida, mas, antes de começar, certifique-se de que o Apache Tomcat está instalado em seu computador.

EU INDICO

Caso você ainda não saiba como configurar o Tomcat no IntelliJ IDEA, assista a este vídeo, que demonstra de forma clara e prática como configurar e utilizar o Tomcat na IDE, o que será essencial para rodar a aplicação com sucesso.

Uma vez que você já possui o Apache Tomcat instalado e configurado no IntelliJ IDEA Ultimate, podemos iniciar o processo de criação do projeto web com suporte a Servlets e JSP. Vamos seguir os seguintes passos:

1. **Criar um novo projeto no IntelliJ:** na tela inicial da IDE, clique em File > New > Project. No menu lateral, selecione a opção Jakarta EE, pois ela já inclui os templates necessários para aplicações web Java.
2. **Definir as configurações do projeto:**
   a) Nome do projeto: escolha um nome como **cadastro**.
   b) *Build System*: selecione Maven.
   c) *Template*: selecione *Web application* (isso incluirá o web.xml, index.jsp e configuração básica).

d) *Application Server*: selecione o Apache Tomcat (versão 9 ou superior). Se ainda não tiver adicionado à IDE, clique em *New*... e localize a pasta onde o Tomcat foi instalado.
e) JDK: selecione uma versão compatível, como o Java 22 (ou uma versão LTS estável).
f) *Group* e *Artifact*: defina algo como org.example e cadastro, respectivamente.

3. **Escolher as dependências:** na próxima tela, marque a dependência Servlet dentro das implementações. Essa dependência permitirá que sua aplicação utilize classes como HttpServlet, HttpServletRequest e HttpServletResponse.
4. **Finalizar criação do projeto:** clique em Create. O IntelliJ criará a estrutura do projeto, incluindo as pastas src/main/java para suas classes Java e src/main/webapp para suas páginas JSP e arquivos estáticos.

EU INDICO

Se você tiver dúvidas ou quiser aprofundar o entendimento sobre como criar e executar aplicações web com Jakarta EE no IntelliJ IDEA, recomendo o tutorial da JetBrains. Ele apresenta, passo a passo e com imagens, como configurar seu ambiente e iniciar sua primeira aplicação web com Servlets e JSP.,

Com o projeto criado e configurado, o próximo passo é entender como ele está organizado.

Estrutura do projeto criado

Após a criação do projeto no IntelliJ IDEA Ultimate, temos uma estrutura inicial automática que já prepara o ambiente para o desenvolvimento web com Servlets e JSP. Essa organização separa claramente os arquivos de código, as configurações e os recursos de interface (Caelum, 2021).

A Figura 3 ilustra como o projeto está organizado. Dentro da pasta src/main/java, localiza-se o pacote de classes Java, onde serão implementadas as funcionalidades da aplicação.

Já na pasta src/main/webapp, temos os arquivos que compõem a interface com o usuário e a pasta WEB-INF, onde está o arquivo de configuração web.xml.

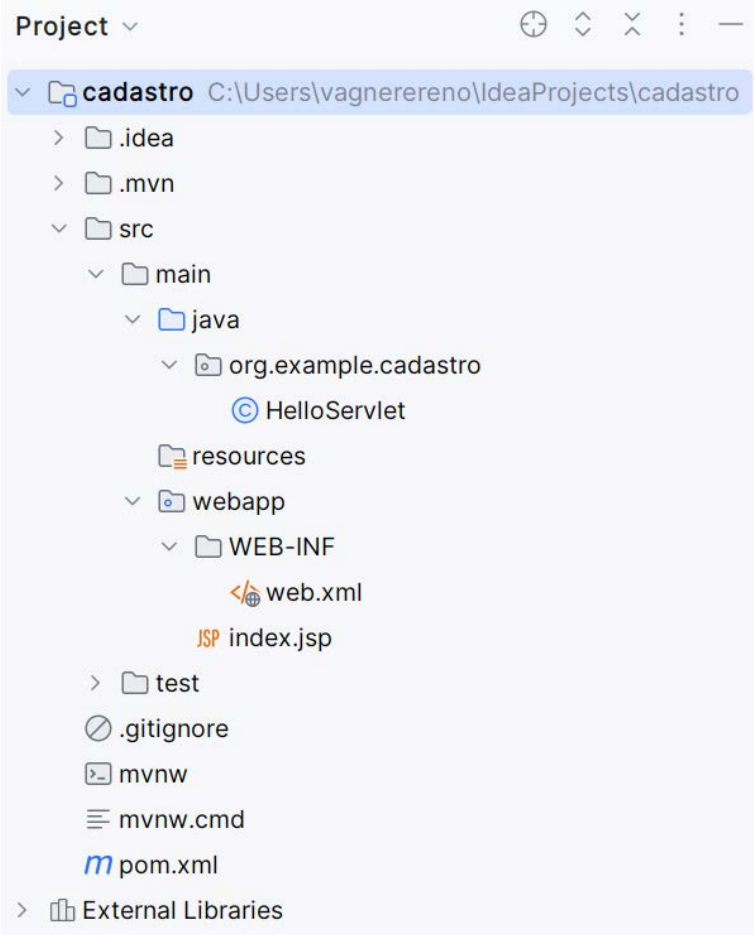


Figura 3 – Estrutura inicial do projeto Java Web. / Fonte: o autor.

Descrição da Imagem: a imagem exibe o painel de projeto do IntelliJ IDEA Ultimate. O projeto principal, chamado cadastro, contém subpastas como .idea, .mvn e src. Dentro de src/main/java, há um pacote org.example.cadastro com uma classe HelloServlet. Em src/main/webapp, aparece a pasta WEB-INF com o arquivo web.xml e um arquivo index.jsp. A raiz do projeto contém também o pom.xml, arquivos do Maven e pastas auxiliares como test e resources. Fim da descrição.

Essa estrutura segue o padrão recomendado para aplicações Java Web baseadas em Jakarta EE com Maven (Eclipse Foundation, 2022).

Criando a primeira página JSP

Vamos começar a construir a aplicação desenvolvendo a interface inicial que o usuário verá ao acessar o sistema. Essa página será escrita em JSP (*JavaServer Pages*), mas ainda conterá apenas elementos estáticos, como um título e um formulário simples para entrada de dados (Murach; Urban, 2014).

Para isso, abra o arquivo chamado index.jsp dentro da pasta src/main/webapp. Esse arquivo será a porta de entrada da aplicação, ou seja, será exibido no navegador quando o projeto for iniciado. O código a seguir apresenta um exemplo básico de conteúdo para essa página:

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<html>
<head>
  <title>Bem-vindo!</title>
</head>
<body>
    <h2>Digite seu nome:</h2>
    <form action="hello-servlet" method="post">
        <input type="text" name="nome" />
        <input type="submit" value="Enviar" />
  </form>
</body>
</html>
```

Nesse exemplo, usamos uma estrutura básica de HTML combinada com a diretiva JSP **<%@ page ... %>**. O formulário é configurado para enviar os dados ao Servlet mapeado como "hello-servlet", utilizando o método POST.

APROFUNDANDO

Essa estrutura já demonstra como o JSP pode funcionar como uma página comum de HTML, mas com a possibilidade de incluir lógica Java — o que será feito mais adiante, ao apresentar dados personalizados com base na entrada do usuário.

A seguir, daremos continuidade ao desenvolvimento, integrando o JSP com um Servlet para criar uma resposta dinâmica.

Criando o Servlet para processar a requisição

Agora que temos a interface preparada com a página JSP, o próximo passo é programar o componente que fará o processamento da entrada enviada pelo usuário. Esse papel é desempenhado pelo Servlet, uma classe Java que roda no servidor e lida diretamente com as requisições HTTP (Murach; Urban, 2014).

No IntelliJ, dentro da pasta src/main/java, localize o pacote "org.example.cadastro". Ao criar o projeto Jakarta EE no IntelliJ IDEA, uma classe chamada **HelloServlet** foi gerada automaticamente. Vamos utilizá-la como base para implementar nossa lógica de processamento, aproveitando sua estrutura já integrada ao ciclo de vida da aplicação web.

```
package org.example.cadastro;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

import java.io.IOException;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse respon-
se)
            throws ServletException, IOException {

        String nome = request.getParameter("nome");

        request.setAttribute("nomeUsuario", nome);

        RequestDispatcher dispatcher = request.getRequestDispatcher("saudacao.jsp");
        dispatcher.forward(request, response);
    }
}
```

Nesse código, definimos que o Servlet responderá a requisições enviadas para a URL **/hello-servlet**, conforme especificado pela anotação @WebServlet.

O método doPost é sobrescrito para interceptar os dados do formulário (neste caso, o nome do usuário), por meio do método getParameter. Esse valor é então armazenado como um atributo da requisição com o nome nomeUsuario.

Em seguida, a requisição é encaminhada internamente para a página saudacao.jsp utilizando um RequestDispatcher, que permite transferir o controle da execução sem encerrar a resposta.

Esse fluxo é essencial para separar a lógica de controle (Servlet) da apresentação (JSP), promovendo uma arquitetura mais clara e manutenível.

Exibindo a saudação personalizada

Depois que o Servlet processa os dados recebidos do formulário, ele repassa essas informações para uma nova página JSP (Murach; Urban, 2014). Essa página será responsável por apresentar a resposta personalizada, neste caso, uma saudação com o nome informado pelo usuário.

Vamos criar um novo arquivo chamado saudacao.jsp dentro da pasta webapp. Essa página acessará o atributo da requisição definido pelo Servlet e o exibirá diretamente na interface.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <title>Saudação</title>
</head>
<body>
    <h2>Olá, ${nomeUsuario}!</h2>
            <p>Seja bem-vindo à nossa aplicação web com Servlet e JSP.</p>
</body>
</html>
```

O destaque aqui está no uso da *Expression Language* (EL), a abordagem moderna que vimos anteriormente. A expressão ${nomeUsuario} acessa diretamente o atributo enviado pelo Servlet, mantendo a página JSP limpa e focada apenas

na exibição dos dados. Esse valor é renderizado dinamicamente, criando uma resposta personalizada para cada usuário.

Com isso, a aplicação agora está completa: o usuário preenche o formulário em **index.jsp**, os dados são processados por **HelloServlet.java** e o resultado é exibido em **saudacao.jsp**.

Visualizando o Resultado no Navegador

Para que tudo funcione corretamente, é essencial garantir que a versão do servidor Tomcat seja compatível com a versão da especificação Jakarta Servlet utilizada no projeto. No exemplo apresentado, utilizamos o **Tomcat 10.1** em conjunto com a **Jakarta Servlet 6**. No entanto, essas versões podem evoluir ao longo do tempo. Por isso, recomenda-se sempre verificar a compatibilidade entre o servidor de aplicações e a versão das bibliotecas utilizadas no pom.xml, conforme a documentação oficial mais recente (Caelum, 2021).

Com o ambiente devidamente configurado, execute a aplicação clicando no botão de execução (*run*) no IntelliJ IDEA. Certifique-se de que a configuração esteja vinculada ao servidor Tomcat e ao artefato ***cadastro:war exploded***. Em seguida, abra o navegador e acesse o endereço "http://localhost:8080/cadastro_war_exploded/". A primeira tela apresentada será a página com o formulário, conforme ilustrado na Figura 4.

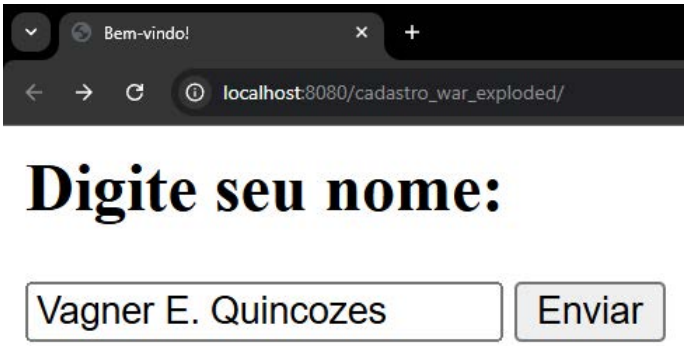


Figura 4 – Página inicial com formulário de entrada. / Fonte: o autor.

Descrição da Imagem: a imagem ilustra a página inicial da aplicação, exibindo um campo de texto para digitar o nome e um botão de envio. Fim da descrição.

Após digitar seu nome e clicar em "Enviar", o navegador será redirecionado para a página de saudação personalizada, conforme ilustrado na Figura 5.

Figura 5 – Página de saudação personalizada. / Fonte: o autor.

Descrição da Imagem: a imagem mostra a tela de resposta da aplicação com a saudação "Olá, Vagner E. Quincozes! Seja bem-vindo à nossa aplicação web com Servlet e JSP". Fim da descrição.

Com isso, você concluiu a criação de uma aplicação Java Web simples, que demonstra como os Servlets e as páginas JSP podem trabalhar em conjunto para processar dados e gerar uma resposta personalizada. Compreender bem esse fluxo de requisição e resposta entre o navegador, o Servlet e as páginas JSP é fundamental, pois ele servirá de base para construções mais complexas.

SESSÕES E COOKIES EM APLICAÇÕES JAVA WEB

O protocolo HTTP é "sem estado" (*stateless*). Isso significa que, a cada nova requisição, o servidor não tem memória de interações anteriores com um mesmo cliente. Para contornar essa limitação e preservar o contexto entre as interações — como manter um usuário logado ou lembrar produtos em um carrinho de compras —, as aplicações Java Web utilizam sessões e cookies.

O protocolo HTTP é "sem estado" (*stateless*)

Esses dois mecanismos mantêm dados associados ao usuário, mas operam de maneiras distintas (Caelum, 2021):

- **Sessões (*server-side*)**: são gerenciadas no lado do servidor. Ao criar uma sessão, o servidor armazena os dados e os associa a um identificador único (ID da sessão). Esse ID é enviado ao cliente (geralmente através de um cookie) e usado para recuperar o contexto a cada nova requisição. Por manter os dados no servidor, as sessões são mais seguras e ideais para informações sensíveis, como dados de login.
- **Cookies (*client-side*)**: são pequenos arquivos de texto armazenados no navegador do cliente. O servidor envia o cookie e o navegador o anexa automaticamente às requisições futuras para o mesmo domínio. São úteis para dados não sensíveis que precisam persistir por longos períodos, como preferências de idioma ou temas.

EU INDICO

Explore o Eu Indico para visualizar de forma dinâmica os conceitos de Servlets, JSPs e controle de sessão! Assista e fortaleça seu entendimento com animações explicativas envolventes.

Agora que você compreende as diferenças conceituais entre sessões e cookies, é hora de explorar como esses mecanismos são implementados na prática em aplicações Java Web.

Trabalhando com sessões em aplicações Java Web

Em aplicações Java Web, a criação e o gerenciamento de sessões são realizados por meio da interface HttpSession, que faz parte da especificação Jakarta Servlet. Toda vez que um cliente realiza uma requisição ao servidor, podemos obter (ou criar) uma sessão associada a ele utilizando o método **request.getSession()** (Caelum, 2021).

Se já existir uma sessão válida para aquele usuário, o método **getSession()** a retorna. Caso contrário, uma nova sessão é criada automaticamente. Essa sessão permanece ativa enquanto o usuário interagir com a aplicação ou até que o tempo máximo de inatividade seja atingido.

Veja um exemplo básico de como criar uma sessão e armazenar um atributo:

```
HttpSession sessao = request.getSession()
sessao.setAttribute("nomeUsuario", "Vagner");
```

Neste trecho, é criada (ou recuperada) uma sessão associada ao usuário, e nela é armazenado um atributo chamado "nomeUsuario", com o valor "Vagner". Esse dado poderá ser acessado em qualquer ponto da aplicação enquanto a sessão estiver ativa (Caelum, 2021).

Mais adiante, para recuperar esse valor em outra parte da aplicação (por exemplo, em uma página JSP), usamos a *Expression Language* (EL) para acessar diretamente o escopo da sessão:

```
${sessionScope.nomeUsuario"}
```

Note que usamos sessionScope, que é um objeto padrão do EL para acessar a sessão, e não o nome da variável sessao que usamos no Servlet.

Além de armazenar dados com setAttribute, o desenvolvedor pode controlar o tempo de vida da sessão (Caelum, 2021). Isso é feito por meio do método setMaxInactiveInterval, que define o tempo (em segundos) de inatividade permitido antes que a sessão expire automaticamente:

```
HttpSession sessao = request.getSession();
sessao.setMaxInactiveInterval(900); // 15 minutos
```

Esse controle é essencial para a segurança da aplicação, especialmente em ambientes onde há dados sensíveis ou múltiplos usuários ativos. Caso seja necessário encerrar a sessão manualmente (por exemplo, ao implementar uma funcionalidade de logout), basta utilizar: "sessao.invalidate();".Com isso, todos os atributos

da sessão são removidos e o usuário deixa de estar associado àquela instância de navegação (Caelum, 2021). O uso adequado de sessões é fundamental para manter o estado da aplicação e garantir uma experiência contínua para o usuário.

Trabalhando com cookies em aplicações Java Web

Para implementar cookies em aplicações Java Web, utilizamos a classe Cookie, disponível na especificação Jakarta Servlet (Caelum, 2021). Ela permite criar, configurar e adicionar cookies à resposta enviada ao navegador do usuário. Veja um exemplo básico:

```
Cookie idioma = new Cookie("idioma", "pt-BR");
idioma.setMaxAge(60 * 60 * 24 * 7); // 7 dias
response.addCookie(idioma);
```

Esse trecho cria um cookie chamado "idioma", com valor "pt-BR", e o configura para expirar em sete dias. Ele é então enviado ao navegador, que armazenará localmente o dado.

Nas próximas requisições, esse cookie será automaticamente enviado ao servidor. Para recuperá-lo, usamos o método request.getCookies():

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("idioma")) {
            String valor = c.getValue();
            // Usar o valor como necessário
        }
    }
}
```

Esse código percorre os cookies recebidos e recupera o valor associado à chave "idioma". Essa abordagem permite personalizar a interface, lembrar preferências do usuário ou controlar comportamentos persistentes.

O uso estratégico de sessões e cookies permite construir aplicações Java Web mais robustas, adaptadas ao comportamento e às necessidades dos usuários.

EM FOCO

Confira a videoaula que preparamos sobre este conteúdo e aprofunde sua compreensão sobre Java Web.

NOVOS DESAFIOS

Ao longo deste estudo, você explorou os primeiros passos para tornar suas aplicações Java acessíveis pela web, compreendendo como os Servlets lidam com requisições e respostas e como as páginas JSP podem gerar conteúdo dinâmico adaptado ao usuário. Mais do que aprender conceitos isolados, você construiu uma base para pensar em aplicações web completas, que conversam com o navegador e respondem de forma eficiente às ações do usuário.

Na prática profissional, esse conhecimento permite que você crie interfaces interativas e responsivas, mantenha o estado da aplicação entre diferentes requisições e ofereça experiências mais fluídas a quem interage com seus sistemas. Compreender o ciclo de vida dos Servlets e saber quando utilizar páginas JSP abre um leque de possibilidades na construção de sistemas corporativos, painéis administrativos, portais internos e outras aplicações amplamente utilizadas no mercado.

Se você chegou até aqui, já percebeu que os desafios de um sistema web vão além do que é visível na tela. A lógica do servidor, a manipulação de sessões, a organização do código e a separação entre responsabilidades são elementos que precisam caminhar juntos para garantir robustez e escalabilidade. Dominar esses fundamentos permite compreender melhor o funcionamento interno das aplicações web e identificar soluções mais adequadas para diferentes contextos profissionais.

A cada nova camada de abstração, seu papel como desenvolvedor também evolui. Ao combinar domínio técnico com boas práticas de arquitetura, você se torna capaz de projetar soluções mais seguras, manuteníveis e alinhadas às demandas do mundo real. Que tal refletir, desde já, sobre como aplicar esses aprendizados em um projeto seu ou dentro de sua equipe?

ATIVIDADES

1. O ciclo de vida de um Servlet é um dos conceitos fundamentais da plataforma Jakarta EE, pois define como o contêiner web gerencia as instâncias das classes Servlet para garantir eficiência e performance. Diferente de um programa Java convencional, um Servlet não é executado e encerrado a cada interação; ele é mantido em memória pelo contêiner para responder a múltiplas requisições de forma otimizada.

   Com base nos conceitos do ciclo de vida de um Servlet, avalie as afirmativas a seguir:

   I - O método init() de um Servlet é invocado pelo contêiner uma única vez, durante a inicialização, e serve para configurar recursos que serão utilizados ao longo de sua vida útil.
   II - Para cada nova requisição HTTP recebida, o contêiner cria uma nova instância da classe do Servlet para garantir o isolamento entre as requisições dos usuários.
   III - O método service() é chamado a cada requisição e é responsável por despachá-la para métodos mais específicos, como doGet() ou doPost(), com base no método HTTP utilizado pelo cliente.
   IV - O método destroy() é executado quando o contêiner decide remover o Servlet da memória, como durante o desligamento da aplicação, permitindo a liberação de recursos.

   É correto o que se afirma em:

   a) I e II, apenas.
   b) II e III, apenas.
   c) I e IV, apenas.
   d) I, III e IV, apenas.
   e) I, II, III e IV.

2. No desenvolvimento de aplicações web com Java, uma boa prática é separar as responsabilidades: o Servlet fica encarregado de receber as requisições e processar a lógica de negócio, enquanto a página JSP se concentra em exibir a resposta visualmente para o usuário. Para que isso funcione, a especificação Java Servlet fornece mecanismos para que o Servlet possa encaminhar dados de forma eficiente para a página JSP.

   Considere o seguinte cenário: um **BuscaServlet** processa uma pesquisa, calcula o número de resultados e precisa exibir uma mensagem em uma página chamada **resultados.jsp**. Para isso, o desenvolvedor escreve o seguinte código no final do método **doGet()** do Servlet:

```
String mensagem = "5 produtos encontrados";
request.setAttribute("totalResultados", mensagem);
RequestDispatcher dispatcher = request.getRequestDispatcher("resultados.jsp");
dispatcher.forward(request, response);
```

Qual alternativa contém o código correto a ser inserido na página resultados.jsp para que ela possa exibir a mensagem definida no Servlet?

a) ${totalResultados}.
b) ${param.totalResultados}
c) <%= request.getParameter("totalResultados") %>.
d) ${sessionScope.totalResultados}.
e) <%= totalResultados %>.

3. O protocolo HTTP, por natureza, não mantém estado (stateless), o que significa que cada requisição é tratada pelo servidor como um evento isolado, sem memória de interações anteriores. Para criar experiências de usuário contínuas, como manter um login ou salvar preferências, as aplicações web precisam de mecanismos para gerenciar o estado. A especificação Java Servlet oferece duas abordagens principais para isso: Sessões (HttpSession) e Cookies.

Considere o seguinte cenário: uma aplicação web precisa gerenciar duas informações de um usuário: o tema visual preferido ('claro' ou 'escuro') e o ID numérico do usuário após ele realizar o login. Os requisitos são:

- O **tema visual** deve ser lembrado por meses, mesmo que o usuário feche e reabra o navegador.
- O **ID do usuário**, por ser um dado sensível, deve ser armazenado de forma segura no servidor e ser automaticamente invalidado após 30 minutos de inatividade.

Considerando as boas práticas de segurança e performance, qual abordagem descreve a maneira correta de armazenar o tema visual e o ID do usuário, respectivamente?

a) Utilizar um Cookie para o tema e a HttpSession para o ID do usuário.
b) Utilizar a HttpSession para o tema e um Cookie para o ID do usuário.
c) Utilizar a HttpSession para armazenar ambos, configurando tempos de expiração diferentes.
d) Utilizar Cookies para armazenar ambos, pois os dados pertencem ao mesmo usuário.
e) Utilizar um Cookie para o tema e um atributo da requisição (request.setAttribute) para o ID do usuário.

REFERÊNCIAS

CAELUM. **Java para desenvolvimento web**. São Paulo: Casa do Código, 2021.

ECLIPSE FOUNDATION. **Jakarta Servlet 6.0**. Version 6.0. Jakarta: Eclipse Foundation, 2022. Disponível em: https://jakarta.ee/specifications/servlet/6.0/. Acesso em: 15 jul. 2025.

JUNEAU, J. Servlets and JavaServer Pages. *In*: **Jakarta EE recipes**: A problem-solution approach. Berkeley, CA: Apress, 2020. p. 1-93.

MURACH, J.; URBAN, M. **Murach's Java servlets and JSP**. Fresno, CA: Mike Murach & Associates, 2014.

WOLF, J. **HTML & CSS**: The comprehensive guide to excelling in HTML5 and CSS3 for responsive web design, dynamic content, and modern layouts. 1. ed. Bonn: Rheinwerk Computing, 2023.

GABARITO

1. Alternativa D.

   Afirmativa I – Correta. O método init() é o ponto de partida do Servlet e a especificação garante que ele será chamado apenas uma vez, antes de qualquer requisição ser processada. É o local ideal para inicializar configurações pesadas ou conexões. Afirmativa II – Incorreta. Esta é uma das características mais importantes do ciclo de vida. Por padrão, o contêiner cria uma única instância do Servlet que atende a múltiplas requisições em threads separadas. Isso economiza memória e melhora a performance, mas exige atenção do desenvolvedor com a concorrência. Afirmativa III – Correta. O método service() funciona como o ponto central de entrada para as requisições. Ele analisa o tipo de método HTTP (GET, POST, PUT, etc.) e invoca o método do...() correspondente, organizando o fluxo de processamento. Afirmativa IV – Correta. O método destroy() é a contraparte do init(). Ele oferece uma oportunidade para o Servlet "limpar a casa", liberando de forma ordenada quaisquer recursos que tenha alocado (como fechar conexões com o banco de dados) antes de ser descarregado da memória.

2. Alternativa A.

   ${totalResultados} – Correta. A *Expression Language* (EL), com a sintaxe ${...}, é a forma moderna e recomendada para acessar atributos definidos no servidor. Ao usar ${totalResultados}, o EL automaticamente procura por um atributo com esse nome no escopo da requisição (onde foi definido pelo Servlet com request.setAttribute()) e exibe seu valor.

   ${param.totalResultados} – Incorreta. O objeto implícito para na EL é usado para acessar parâmetros que vêm do cliente (enviados em um formulário ou na URL), e não atributos que foram definidos no lado do servidor.
   <%= request.getParameter("totalResultados") %> – Incorreta. O método request.getParameter() também serve para recuperar parâmetros da requisição vindos do cliente. O dado "totalResultados" foi adicionado como um atributo no servidor, que deve ser recuperado com request.getAttribute() (na sintaxe antiga) ou, preferencialmente, com EL.

   ${sessionScope.totalResultados} – Incorreta. O escopo sessionScope é específico para acessar atributos armazenados na sessão do usuário (session.setAttribute()). No cenário descrito, o dado foi armazenado apenas no escopo da requisição, que tem uma vida útil mais curta, durando apenas até a resposta ser enviada ao cliente.

   <%= totalResultados %> – Incorreta. A sintaxe de expressão do JSP (<%= ... %>) exige uma variável ou um método Java válido em seu interior. A variável de nome totalResultados não foi declarada na página JSP, então este código resultaria em um erro de compilação da página.

3. Alternativa A.

Utilizar um Cookie para o tema e a HttpSession para o ID do usuário. – Correta. Esta é a solução ideal. O Cookie é perfeito para armazenar dados pequenos, não sensíveis e de longa duração no cliente (persistência), como a preferência de um tema. A HttpSession é a escolha segura para armazenar dados críticos e do lado do servidor, como o ID de um usuário logado, com um ciclo de vida controlado pela atividade e timeouts.

Utilizar a HttpSession para o tema e um Cookie para o ID do usuário. – Incorreta. Esta opção inverte perigosamente a lógica. Armazenar o ID do usuário em um Cookie é uma falha de segurança, pois o dado fica exposto e pode ser manipulado no cliente. Armazenar o tema na sessão não atenderia ao requisito de persistência por meses, pois a sessão é perdida quando o navegador é fechado.

Utilizar a HttpSession para armazenar ambos, configurando tempos de expiração diferentes. – Incorreta. Embora a sessão pudesse armazenar o tema, ela não atenderia ao requisito de persistência de longo prazo (meses), pois o ciclo de vida da sessão é muito mais curto e atrelado à navegação ativa.

Utilizar Cookies para armazenar ambos, pois os dados pertencem ao mesmo usuário. – Incorreta. Armazenar o ID de um usuário logado em um Cookie é uma prática de segurança muito ruim. Dados sensíveis que validam a identidade de um usuário devem sempre ser mantidos no lado do servidor.

Utilizar um Cookie para o tema e um atributo da requisição (request.setAttribute) para o ID do usuário. – Incorreta. Atributos da requisição (request.setAttribute) são stateless e existem apenas durante o ciclo de vida de uma única requisição HTTP. O ID do usuário seria perdido assim que a resposta do login fosse enviada, não servindo para manter o estado entre diferentes páginas.

UNIDADE 3

JAVASERVER FACES E PADRÕES DE PROJETO MVC

MINHAS METAS

- Conhecer os fundamentos e a arquitetura do framework JSF.
- Aplicar o padrão de projeto MVC em aplicações Java Web.
- Construir interfaces web com componentes e validações em JSF.
- Desenvolver um Controller (Bean) para gerenciar dados e ações.

INICIE SUA JORNADA

Imagine que você é responsável por desenvolver uma aplicação web para uma instituição de ensino. O sistema precisa permitir que estudantes se inscrevam em disciplinas, atualizem seus dados cadastrais e visualizem suas notas, tudo por meio de uma interface acessível, com boa usabilidade, mantendo a separação entre a lógica do sistema e o que é exibido ao usuário.

Esse tipo de demanda é cada vez mais comum no mercado de desenvolvimento web. Grandes empresas e organizações públicas exigem soluções robustas, organizadas e sustentáveis, que sejam também fáceis de manter e evoluir ao longo do tempo. Nessa perspectiva, conhecer frameworks que promovem boas práticas de desenvolvimento e incentivam a reutilização de componentes pode ser um diferencial decisivo na sua carreira.

Pense em uma aplicação que exige validação de formulários, navegação entre várias telas, comunicação com o banco de dados e componentes interativos como tabelas dinâmicas e gráficos. Com o uso de frameworks como o ***JavaServer Faces*** (**JSF**), essas tarefas deixam de ser desafios isolados e passam a fazer parte de um fluxo bem estruturado. Você poderá experimentar isso criando interfaces reais, manipulando dados e organizando seu código de forma clara e eficaz.

Ao longo do seu aprendizado, reflita sobre como as tecnologias utilizadas impactam não só o funcionamento do sistema, mas também a experiência do usuário, a organização da equipe de desenvolvimento e a qualidade do software entregue. A jornada que se inicia agora é uma oportunidade para desenvolver não apenas habilidades técnicas, mas também um olhar estratégico sobre como construir aplicações web modernas, modulares e alinhadas às demandas do mundo profissional.

PLAY NO CONHECIMENTO

Enquanto você estuda os fundamentos do JSF, que tal ter uma visão do futuro? Em nosso podcast, discutimos temas como persistência de dados e segurança, que complementam seu aprendizado. Dê o play e enriqueça sua jornada!

DESENVOLVA SEU POTENCIAL

JAVASERVER FACES (JSF)

O ***JavaServer Faces***, conhecido pela sigla **JSF**, é um framework da plataforma Java voltado ao desenvolvimento de interfaces web dinâmicas com base em componentes reutilizáveis. Criado para simplificar a construção de aplicações mais complexas, o JSF adota uma estrutura clara que valoriza a organização, a produtividade e o uso de boas práticas no desenvolvimento web corporativo.

A principal proposta do JSF é permitir que desenvolvedores Java construam interfaces web de forma mais estruturada e reutilizável, com suporte automático a validação de dados, controle de navegação, injeção de dependências e separação entre as camadas de visualização, controle e lógica de negócios. Tudo isso é viabilizado por meio de uma abordagem declarativa e de um ciclo de vida bem definido, que oferece maior previsibilidade e manutenção do código (Juneau, 2020).

O que motivou a criação do JSF

Antes do surgimento do JSF, o desenvolvimento de aplicações web em Java envolvia basicamente o uso de **Servlets e JSP** (*JavaServer Pages*). Embora essas tecnologias tenham permitido grandes avanços na web dinâmica, com o tempo elas começaram a mostrar limitações importantes (Algaworks, 2015).

APROFUNDANDO

O que havia de errado com Servlets e JSP?
Em um primeiro momento, os Servlets eram responsáveis por processar requisições e gerar as respostas em código Java. Isso significava escrever HTML manualmente dentro de métodos como **doGet()** e **doPost()**, o que tornava o código extenso, difícil de manter e propenso a erros. Para amenizar esse problema, surgiram as JSPs, permitindo escrever HTML com pequenos trechos de código Java embutido. No entanto, essa prática levou ao efeito contrário: a lógica de negócio passou a invadir a camada de apresentação, gerando alto acoplamento entre o visual e o processamento da aplicação.

À medida que as aplicações se tornaram mais interativas e com maior número de páginas, formulários e regras, cresceu a necessidade de uma abordagem mais moderna e orientada a componentes, como já ocorria em outras plataformas como ASP.NET (Microsoft) ou o próprio modelo proposto pela especificação Java EE (Mann, 2005). Foi nesse contexto que o JSF surgiu como uma resposta oficial para:

- Separar de forma clara a lógica de negócio da interface de usuário.
- Facilitar a criação de componentes reutilizáveis.
- Oferecer um ciclo de vida completo para as requisições, com suporte nativo a validação, conversão, mensagens e controle de navegação.
- Adotar o padrão de projeto MVC (*Model-View-Controller*) de forma natural, mantendo a coesão da aplicação.
- Incentivar boas práticas no desenvolvimento web corporativo.

Para atender a esses objetivos, o JSF incorporou uma série de inovações que o diferenciam das soluções anteriores:

ABSTRAÇÃO POR COMPONENTES

JSF adota uma abordagem baseada em componentes reutilizáveis, como <h:form> e <h:dataTable>, facilitando a construção de interfaces de forma estruturada e modular.

GERENCIAMENTO DO CICLO DE VIDA DA INTERFACE
O framework cuida automaticamente do ciclo completo da requisição, desde a submissão até a renderização da resposta, o que reduz a complexidade do código.
VALIDAÇÃO E CONVERSÃO EMBUTIDAS
Validações e conversões de dados são configuradas declarativamente, sem necessidade de lógica adicional em cada tela, aumentando a produtividade e consistência.
CONTROLE COM BEANS GERENCIADOS
JSF utiliza beans Java como controladores (*backing beans*), organizando a lógica com escopos bem definidos (@RequestScoped, @SessionScoped etc.).
SUPORTE AO PADRÃO MVC NATIVO
A separação entre visual (*Facelets*), controle (*beans*) e modelo (entidades) é clara e bem definida, incentivando boas práticas de arquitetura
TRATAMENTO DE EVENTOS E AÇÕES SIMPLIFICADO
É possível vincular ações diretamente aos componentes da interface, como action="#{usuarioBean.salvar()}", promovendo coesão e clareza.

Além disso, a substituição do JSP pelo Facelets como motor de visualização trouxe avanços significativos, como suporte a templates, inclusão de fragmentos com <ui:include> e composição de páginas reutilizáveis com <ui:composition> (Burns; Griffin, 2010).

Esses recursos modernizam a camada de apresentação e tornam o desenvolvimento mais limpo, produtivo e alinhado aos princípios de componentização.

O padrão MVC por trás do JSF

Antes de explorar os detalhes técnicos do *JavaServer Faces*, é importante compreender a base arquitetural sobre a qual o *framework* se apoia: o padrão *Model-View-Controller*, mais conhecido como MVC (Dathan; Ramnath, 2025).

EU INDICO

Para quem quiser entender de forma rápida o padrão MVC, este vídeo explica em 3 minutos como Model, View e Controller se organizam para separar lógica, interface e controle, facilitando o desenvolvimento de aplicações de software.

O MVC é uma abordagem de organização de aplicações que separa claramente as responsabilidades em três partes (Dathan; Ramnath, 2025):

- ***Model* (modelo)**: é onde ficam os dados e regras de negócio da aplicação. Essa camada trata da lógica que sustenta o sistema.
- ***View* (visão)**: é a camada visual, responsável por exibir as informações ao usuário e por capturar suas interações, como cliques ou preenchimento de formulários.
- ***Controller* (controlador)**: atua como intermediário entre a visão e o modelo, interpretando as ações do usuário e coordenando as atualizações de dados e da interface.

Essa divisão permite construir sistemas modulares, facilitando a manutenção e a colaboração entre diferentes profissionais. A Figura 1 ilustra os componentes do padrão MVC.

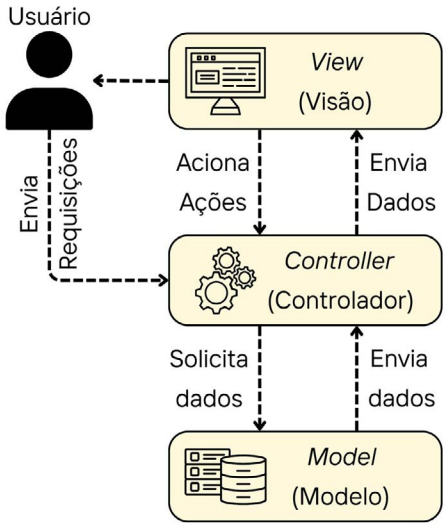


Figura 1 – Padrão de arquitetura *Model-View-Controller* (MVC) / Fonte: o autor.

Descrição da Imagem: a imagem mostra um diagrama da arquitetura MVC organizado em três retângulos empilhados verticalmente. De cima para baixo, os retângulos são: "View (Visão)", "Controller (Controlador)" e "Model (Modelo)". Um ícone de Usuário, à esquerda, inicia o fluxo enviando uma requisição para o Controller (o retângulo do meio). O Controller solicita dados ao Model (o retângulo inferior) e, ao recebê-los, os envia para a View (o retângulo superior). A View retorna a informação ao Usuário e também pode acionar ações de volta para o Controller. Fim da descrição.

EU INDICO

Que tal ver o fluxo do MVC em movimento? Preparamos um vídeo que anima a jornada completa de uma requisição, mostrando na prática como o Controller, o Model e a View interagem a cada passo, desde o clique do usuário até a resposta

No contexto do JSF, essa estrutura é aplicada de forma bastante natural. As páginas XHTML com tags JSF representam a *View*, permitindo a criação de interfaces interativas. Os controladores, que são classes Java gerenciadas pelo framework, assumem o papel de *Controller*, lidando com eventos e direcionando a lógica da aplicação. Os dados e regras de negócio residem na *Model*, formada por classes Java simples, frequentemente integradas com banco de dados e outras camadas do sistema (Algaworks, 2015).

FUNDAMENTOS DO JSF

O ciclo de vida de uma requisição no JSF, ilustrado na Figura 2, é um dos conceitos centrais que diferencia esse *framework* das abordagens tradicionais. Quando um usuário interage com uma página JSF, por exemplo ao clicar em um botão ou submeter um formulário, o framework segue uma série de fases bem definidas até que a resposta seja gerada e exibida no navegador.

Esse processo automatizado garante consistência na manipulação dos dados, facilita a manutenção do sistema e contribui para a separação das responsabilidades da aplicação (Burns; Griffin, 2010).

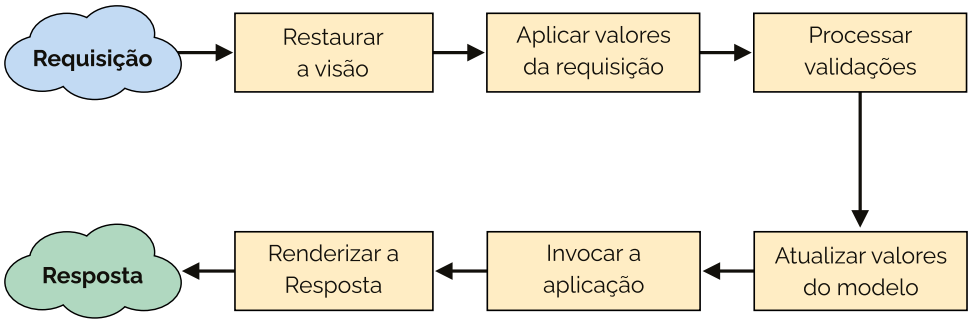


Figura 2 – Ciclo de vida de uma requisição JSF. / Fonte: o autor.

Descrição da Imagem: a figura mostra um fluxograma do ciclo de vida de uma requisição em JSF. O processo inicia com a requisição, passa pelas etapas de restaurar a visão, aplicar valores, validar, atualizar o modelo e invocar a aplicação, e finaliza com a renderização da resposta enviada ao cliente. Fim da descrição.

O ciclo completo pode parecer complexo à primeira vista, mas ele segue uma lógica bastante previsível. Entender essas fases permite ao desenvolvedor intervir nos momentos certos, seja para validar dados, executar ações ou atualizar a interface (Burns; Griffin, 2010). Veja, a seguir, as seis fases principais do ciclo de vida de uma requisição JSF:

RESTORE VIEW (RESTAURAR A VISÃO)

Nessa fase inicial, o JSF tenta reconstruir a árvore de componentes da interface (UI Component Tree) com base na requisição recebida. Se for a primeira vez que a página está sendo acessada, uma nova árvore é criada.

APPLY REQUEST VALUES (APLICAR VALORES DA REQUISIÇÃO)

O JSF percorre os componentes da interface e tenta extrair os dados submetidos (como valores de campos de formulário) para preenchê-los nos respectivos componentes.

PROCESS VALIDATIONS (PROCESSAR VALIDAÇÕES)

Os valores inseridos pelo usuário são validados de acordo com as regras declaradas, como campos obrigatórios, formatos numéricos ou datas. Se houver erro, mensagens são geradas e o fluxo pode retornar à fase de renderização sem continuar para as próximas etapas.

***UPDATE MODEL VALUES* (ATUALIZAR VALORES DO MODELO)**
Os valores validados são finalmente transferidos dos componentes da interface para os atributos correspondentes nos beans de controle (backing beans), atualizando o estado da aplicação.
***INVOKE APPLICATION* (INVOCAR A APLICAÇÃO)**
Essa é a fase onde as ações do usuário são efetivamente executadas — como salvar um registro, realizar um cálculo ou redirecionar para outra página. É aqui que métodos como usuarioBean.salvar() são chamados.
***RENDER RESPONSE* (RENDERIZAR A RESPOSTA)**
Por fim, o JSF monta a página que será exibida ao usuário, com base na árvore de componentes. Essa renderização pode incluir mensagens de erro, dados processados ou redirecionamentos.

Durante esse fluxo, o JSF constrói e manipula uma árvore de componentes, conhecida como component tree, uma estrutura em memória que representa todos os elementos da interface (Burns; Griffin, 2010). Esses componentes não apenas recebem dados e executam ações, mas também lidam com eventos de forma orientada a objetos. Interações como cliques ou envios de formulário geram eventos que são tratados pelos métodos definidos nos backing beans, o que facilita a organização do código e amplia as possibilidades de interatividade, especialmente quando combinados com recursos como <f:ajax>.

Esse fluxo é transparente ao desenvolvedor na maior parte do tempo, mas é possível intervir em fases específicas utilizando interceptadores, validadores personalizados, listeners e anotações apropriadas. Ter essa compreensão amplia significativamente a capacidade de diagnóstico de erros e de implementação de soluções mais robustas e elegantes.

FacesServlet e configuração inicial

Ao iniciar um projeto com JSF, um dos primeiros elementos que precisa ser compreendido é o **FacesServlet**. Esse servlet é o coração do framework: ele é responsável por orquestrar todo o ciclo de vida de uma requisição JSF, desde o

momento em que o usuário envia uma requisição até a renderização da resposta na tela (Mann, 2005).

APROFUNDANDO

FacesServlet é o **controlador central** do **JSF**. Todas as requisições JSF passam por ele, que interpreta as ações do usuário, manipula a árvore de componentes e delega as tarefas para os *beans* e outras partes da aplicação.

O FacesServlet funciona como um *dispatcher* que:

- Mapeia as URLs JSF (geralmente terminadas em .xhtml).
- Processa a árvore de componentes de UI.
- Garante que todas as fases do ciclo de vida sejam executadas na ordem correta.
- Lida com validações, conversões e mensagens.
- Gera a resposta final para o navegador.

Sem ele, a aplicação JSF simplesmente não funciona.

Antes do JSF 2.0, era necessário registrar manualmente o FacesServlet no arquivo web.xml e configurar o mapeamento de URLs (Burns; Griffin, 2010), como no exemplo a seguir:

```
<servlet>
  <servlet-name>Faces Servlet</servlet-name>
  <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>Faces Servlet</servlet-name>
  <url-pattern>*.xhtml</url-pattern>
</servlet-mapping>
```

No entanto, com a evolução da plataforma Java EE e a introdução de anotações, tornou-se possível simplificar esse processo (Burns; Griffin, 2010). A partir do JSF 2.2, muitas configurações podem ser feitas automaticamente pelo contêiner de aplicações (como o WildFly, Payara ou TomEE), desde que o projeto esteja estruturado corretamente.

Mesmo assim, é importante manter o padrão de usar a extensão .xhtml para as páginas e garantir que o FacesServlet seja mapeado para essas requisições.

Um projeto funcional com JSF precisa, basicamente, de: 1) Um arquivo faces-config.xml (obrigatório apenas em versões mais antigas), 2) Um faces servlet devidamente configurado, 3) Páginas .xhtml com as tags JSF, e 4) Backing beans anotados e gerenciados pelo CDI (por exemplo, @Named, @RequestScoped) (Algaworks, 2015).

Em projetos modernos, é comum usar a anotação @FacesConfig para indicar a versão do JSF e garantir a ativação de recursos:

```
@FacesConfig(version = FacesConfig.Version.JSF_2_3)
public class ConfiguracaoJSF {
}
```

Essa classe pode ser deixada vazia, servindo apenas como sinalizador para o contêiner ativar o JSF 2.3 ou superior.

CONSTRUINDO INTERFACES COM JSF

Ao contrário do JSP, o JSF introduz uma forma mais moderna e organizada de construir páginas: os Facelets. Essa tecnologia permite criar interfaces em arquivos .xhtml utilizando tags padronizadas, muitas delas no formato <h:...>, específicas do JSF (Algaworks, 2015). Os Facelets promovem a separação clara entre apresentação e lógica, melhoram a reutilização de componentes e facilitam o uso de templates.

No coração de qualquer página JSF estão algumas tags essenciais, responsáveis por criar formulários, campos de entrada, tabelas de dados e outras estruturas visuais (Juneau, 2020).

A seguir, exploramos três das *tags* mais utilizadas em aplicações JSF:

<H:FORM> – FORMULÁRIO HTML JSF

Todo formulário em JSF deve estar encapsulado dentro de um <h:form>. Essa tag é responsável por preparar os dados que serão enviados e recebidos pela árvore de componentes do JSF. Veja um exemplo:

```
<h:form>
 <h:commandButton value="Enviar" action="#{usuarioBean.salvar}"
/>
</h:form>
```

<H:INPUTTEXT> – CAMPO DE TEXTO VINCULADO AO BEAN

Permite criar campos de entrada de texto que são automaticamente conectados a atributos dos beans de controle por meio de EL (#{}), dispensando o uso direto de request.getParameter(). Veja um exemplo:

```
<h:form>
  <h:inputText value="#{usuarioBean.nome}" />
</h:form>
```

<H:DATATABLE> – TABELA DE DADOS COM ITERAÇÃO

Essa tag exibe listas ou coleções de objetos como uma tabela interativa. É amplamente utilizada para exibir resultados de buscas, cadastros e listas geradas dinamicamente. Veja um exemplo:

```
<h:dataTable value="#{usuarioBean.usuarios}" var="usuario">
  <h:column>
    <f:facet name="header">Nome</f:facet>
   #{usuario.nome}
  </h:column>
</h:dataTable>
```

Essas tags representam o primeiro passo na construção de **interfaces declarativas** com JSF. Ao invés de misturar lógica e visual, como ocorria nos JSPs, o uso dos Facelets com essas tags permite um desenvolvimento mais limpo, organizado e fácil de manter.

Validações e conversões de dados

O JSF oferece suporte nativo para **validações** e **conversões** diretamente nas páginas, sem a necessidade de escrever código Java adicional para casos comuns. Esses recursos contribuem para a robustez da aplicação, garantindo que os dados manipulados estejam sempre em conformidade com as regras esperadas (Juneau, 2020).

As validações em JSF podem ocorrer automaticamente por meio de atributos em componentes ou com o uso de validadores prontos, como required, f:validateLength, f:validateDoubleRange, entre outros. Quando uma validação falha, o JSF interrompe o ciclo de vida na fase de validação e exibe mensagens de erro associadas ao componente.

Para entender como aplicar, na prática, veja um exemplo de campo obrigatório com tamanho mínimo:

```
<h:inputText value="#{usuarioBean.nome}" required="true">
  <f:validateLength minimum="3" />
</h:inputText>
<h:message for="nome" />
```

Você também pode criar validadores personalizados em Java implementando a interface javax.faces.validator.Validator. Isso é útil para regras mais específicas, como CPF válido ou e-mail institucional.

Os conversores de tipo, por outro lado, transformam os dados de entrada (strings) em tipos específicos, como Date, Integer, Double ou mesmo objetos personalizados. Isso é feito na fase de conversão, antes da validação. Veja um exemplo de como aplicar um conversor de data:

```
<h:inputText value="#{eventoBean.data}">
  <f:convertDateTime pattern="dd/MM/yyyy" />
</h:inputText>
```

Para objetos mais complexos (como um Departamento, por exemplo), é comum criar conversores customizados implementando a interface javax.faces.convert.Converter. Eles são essenciais para que o JSF saiba converter de string para objeto e vice-versa ao trabalhar com menus, combos e seleções.

Interatividade com Ajax e bibliotecas como PrimeFaces

Uma das grandes vantagens do JSF em relação a tecnologias mais antigas como Servlets e JSP é a capacidade de adicionar **interatividade** às páginas de forma declarativa e com pouca complexidade. Isso é possível graças à integração com **Ajax** (Silva, 2007) e ao uso de bibliotecas como **PrimeFaces**, que estendem os recursos do JSF com componentes visuais avançados.

O JSF inclui suporte nativo ao Ajax por meio da tag <f:ajax>, que permite executar requisições assíncronas sem recarregar toda a página. Com isso, ações como validar campos, atualizar tabelas ou exibir mensagens podem ocorrer de forma dinâmica, melhorando a experiência do usuário (Silva, 2007). Veja um exemplo prático de como isso funciona:

```
<h:inputText value="#{usuarioBean.nome}">
  <f:ajax event="blur" render="msgNome" />
</h:inputText>
<h:message id="msgNome" for="nome" />
```

Neste exemplo, ao sair do campo de texto (blur), o JSF processa a validação do valor e atualiza apenas a área com a mensagem de erro, sem recarregar a página (Silva, 2007).

Além do Ajax nativo, vale muito a pena conhecer o ***PrimeFaces***. Trata-se de uma biblioteca rica em componentes prontos para uso com JSF, bastante popular em projetos corporativos (Algaworks, 2015). Ela oferece desde calendários, gráficos e tabelas interativas até caixas de diálogo, *uploads* de arquivos e muito mais.

EU INDICO

A documentação mais rica e interativa do PrimeFaces é o seu site de demonstração oficial, o *PrimeFaces Showcase*. Recomendo a consulta a este recurso para visualizar o funcionamento de cada componente, explorar suas funcionalidades e obter exemplos de código-fonte (.xhtml e Java) prontos para implementação em seus projetos.

O uso é bastante simples. Para exibir um calendário, basta usar a seguinte *tag*:

```
<p:calendar value="#{eventoBean.dataEvento}" pattern="dd/MM/yyyy" />
```

Além dos componentes, o PrimeFaces também oferece suporte avançado a temas, responsividade e integração com Ajax de forma ainda mais simples. Para utilizá-lo, é necessário apenas adicionar a dependência no projeto e importar o namespace http://primefaces.org/ui nas páginas .xhtml.

CONTROLADORES E NAVEGAÇÃO COM JSF

Com a interface já estruturada, é hora de entender como o JSF organiza o controle das ações do usuário e o fluxo de navegação entre as páginas da aplicação (Algaworks, 2015).

Managed Beans, escopos e anotações

No *JavaServer Faces*, os *Managed Beans* são peças-chave para o funcionamento da aplicação. Eles representam os objetos de controle, responsáveis por reagir

às ações do usuário e interagir com os componentes visuais das páginas. Em outras palavras, são os "controladores" do padrão MVC, conectando a camada de visualização (*View*) à lógica de negócio (*Model*) (Algaworks, 2015) (Burns; Griffin, 2010).

Essas classes Java são gerenciadas automaticamente pelo contêiner da aplicação, o que significa que seu ciclo de vida — criação, escopo e destruição — é controlado pelo próprio framework (Burns e Griffin, 2010). Com a integração ao Context and Dependency Injection (CDI), os beans passaram a ser anotados com @Named, o que os torna acessíveis nas páginas JSF usando EL (Expression Language), como em #{usuarioBean}.

A anotação que define o escopo do bean é igualmente importante. Ela determina por quanto tempo aquele objeto estará disponível e como será compartilhado entre requisições. Os principais escopos no JSF são:

@REQUESTSCOPED

Válido para uma única requisição HTTP. Indicado para operações pontuais, como o envio de formulários simples.

@VIEWSCOPED

Mantém o *bean* enquanto o usuário estiver na mesma página. Ideal para páginas com interações AJAX.

@SESSIONSCOPED

O bean persiste durante toda a sessão do usuário. Útil para dados como login ou preferências.

@APPLICATIONSCOPED

Compartilhado por todos os usuários e sessões. Recomendado para dados estáticos, como configurações globais.

A seguir, um exemplo prático de bean com escopo de visualização:

```
@Named("usuarioBean")
@ViewScoped
public class UsuarioBean implements Serializable {
   private String nome;

   public void salvar() {
      // lógica de salvar usuário
   }

   // getters e setters
}
```

Vale ressaltar que, quanto mais longo o escopo, maior a permanência do *bean* na memória. Escopos curtos tornam a aplicação mais eficiente, desde que usados com planejamento.

Navegação entre páginas: implícita, explícita e condicional

Em aplicações web, a navegação entre páginas é uma ação recorrente: após o login, um usuário é redirecionado para a área principal; ao concluir um cadastro, pode ser levado a uma tela de confirmação. No *JavaServer Faces*, essa navegação pode ocorrer de diferentes formas, cada uma com suas vantagens. Vamos explorar as abordagens mais utilizadas: navegação **implícita**, **explícita** e **condicional** (Burns; Griffin, 2010).

A **navegação implícita** é a forma mais simples e direta. Ela consiste em retornar, a partir de um método do *bean*, o nome da página de destino (sem a extensão). O JSF interpreta automaticamente esse retorno e redireciona o usuário para o arquivo correspondente, como no exemplo:

```
public String irParaCadastro() {
   return "cadastro";
}
```

Ao retornar "cadastro", o JSF buscará a página cadastro.xhtml e a exibirá ao usuário. Essa abordagem funciona bem quando os nomes das páginas são estáveis e a navegação é linear.

Por outro lado, a **navegação explícita** oferece maior controle ao desenvolvedor. Nesse caso, os caminhos entre páginas são definidos em um arquivo de configuração (normalmente faces-config.xml). Essa estratégia é útil quando se deseja centralizar as regras de navegação, especialmente em projetos maiores ou com múltiplos redirecionamentos:

```
<navigation-rule>
  <from-view-id>/login.xhtml</from-view-id>
  <navigation-case>
    <from-outcome>sucesso</from-outcome>
    <to-view-id>/dashboard.xhtml</to-view-id>
  </navigation-case>
</navigation-rule>
```

Aqui, se um método retornar "sucesso" na página login.xhtml, o usuário será redirecionado para dashboard.xhtml, conforme configurado. Essa abordagem explicita os caminhos, mas exige manutenção adicional sempre que novas rotas são incluídas.

Por fim, a **navegação condicional** permite decidir o destino com base em condições lógicas avaliadas em tempo de execução (Burns; Griffin, 2010). Isso pode ser feito dentro dos métodos Java, usando estruturas condicionais que retornam diferentes resultados conforme o estado da aplicação:

```
public String redirecionar() {
    if (usuario.isAdmin()) {
        return "admin";
    } else {
        return "usuario";
    }
}
```

O método redirecionar() avalia se o usuário é um administrador e, com base nisso, o leva para admin.xhtml ou usuario.xhtml. Essa forma é útil para adaptar a experiência conforme o perfil ou o contexto do usuário.

PROJETO PRÁTICO: APLICAÇÃO DE CADASTRO COM JSF

Após a exploração dos conceitos teóricos e dos componentes fundamentais do *JavaServer Faces*, este é o momento de consolidar o conhecimento adquirido. A seguir, construiremos uma aplicação web de cadastro de ponta a ponta, desenvolvendo um sistema simples para o registro de alunos.

Para isso, utilizaremos um conjunto de ferramentas profissionais, incluindo a IDE IntelliJ IDEA Ultimate e o servidor de aplicação WildFly.

Configurando o ambiente: o servidor de aplicação WildFly

Antes de criar nosso projeto, precisamos de um servidor de aplicação. Ele é o ambiente de execução que entende como receber requisições da internet, processá-las com a tecnologia JSF e devolver uma página como resposta ao navegador do usuário. Para este projeto, utilizaremos o **WildFly**, um servidor robusto e completo, que já vem com todo o suporte necessário para aplicações JSF modernas, simplificando nosso desenvolvimento (Algaworks, 2015).

EU INDICO

Para um guia visual completo sobre como instalar e configurar o WildFly diretamente no IntelliJ IDEA, recomendamos o vídeo a seguir. Ele demonstra de forma clara todo o processo de integração entre a IDE e o servidor.

Você deve baixar o arquivo .zip do WildFly no site oficial: *wildfly.org/downloads/* e, em seguida, seguir o tutorial indicado acima.

Criação e estruturação inicial do projeto

Com o ambiente de desenvolvimento devidamente configurado, nosso próximo passo é criar a estrutura base (o "esqueleto") do projeto no IntelliJ IDEA. O uso de um assistente de projeto, como o oferecido pela IDE, é uma prática recomendada,

pois garante que todas as configurações iniciais e dependências essenciais sejam adicionadas corretamente, evitando erros comuns.

O processo se inicia através do assistente de criação de projetos da IDE. No menu superior, navegue até **File > New > Project....**

Na janela que se abre, a primeira e mais importante escolha é o tipo de projeto. No painel esquerdo, selecione a opção Jakarta EE. Essa seleção informa ao IntelliJ que estamos construindo uma aplicação corporativa, o que habilita todas as ferramentas e configurações automáticas necessárias para o desenvolvimento web com JSF (Algaworks, 2015).

A tela principal de configuração solicitará as seguintes informações:

NAME
Defina um nome para o projeto, como cadastro-alunos.
TEMPLATE
Certifique-se de que a opção *Web application* esteja selecionada.
APPLICATION SERVER
Selecione o servidor WildFly que você configurou no passo anterior.
BUILD SYSTEM
Mantenha a opção Maven, que é o gerenciador de dependências que utilizaremos.

Um dos grandes benefícios de usar este assistente é a gestão de dependências. Após clicar em *Next*, na tela seguinte, localize a seção *Dependencies*. Aqui, garantiremos que a base da nossa aplicação esteja completa, selecionando as **especificações** essenciais: ***Jakarta Servlet***, a base para a comunicação web; ***Jakarta Server Faces* (JSF)**, nosso *framework* de interface; e ***Contexts and Dependency Injection* (CDI)**, o motor que irá gerenciar nossos *Beans* (Algaworks, 2015).

Além das especificações (as "regras"), precisamos selecionar a implementação (o "motor") do JSF. Na subseção *Implementations*, certifique-se de marcar a opção ***Mojarra Server Faces***, que é a implementação de referência oficial. Ao marcar todas essas opções, como ilustrado na Figura 3, o IntelliJ irá adicionar automaticamente as bibliotecas necessárias ao nosso arquivo de configuração pom.xml, poupando-nos do trabalho de configurá-las manualmente.

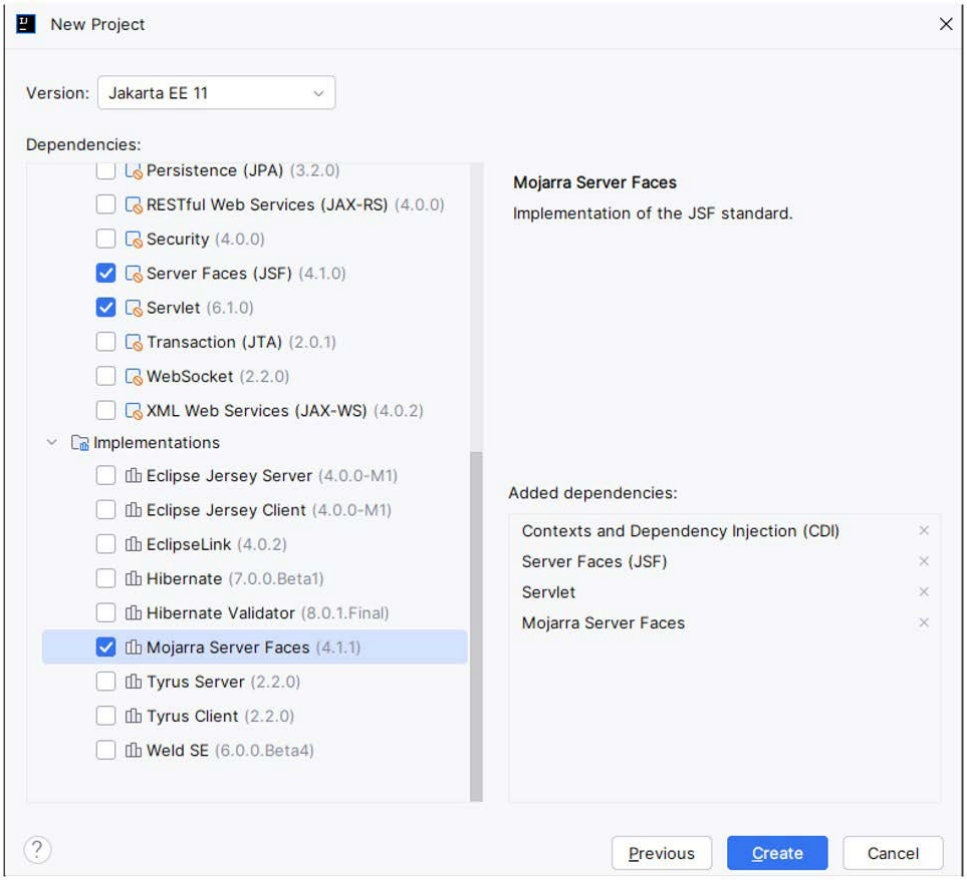


Figura 3 – Seleção de dependências no assistente de projeto / Fonte: o autor.

Descrição da Imagem: tela do IntelliJ IDEA Ultimate mostrando a criação de projeto Jakarta EE 11 com opções de dependências selecionadas. Fim da descrição.

Após preencher todos os campos, clique em ***Create***. A IDE levará alguns instantes para gerar a estrutura de pastas, baixar as dependências iniciais através do Maven e indexar o projeto.

Ao final do processo, teremos um esqueleto de projeto funcional, pronto para que possamos começar a organizar e implementar nossa arquitetura MVC, o que será o foco da próxima seção.

A arquitetura MVC na estrutura do projeto

Após a criação do projeto, o IntelliJ IDEA nos apresenta uma estrutura de diretórios padrão, organizada pelo Maven. Compreender o propósito de cada pasta é o primeiro passo para organizar nosso código de forma limpa, manutenível e, principalmente, seguindo o padrão MVC (Dathan; Ramnath, 2025). A estrutura essencial do nosso projeto cadastro-alunos será a seguinte:

```
cadastro-alunos

src
main
java -> (Camadas Model e Controller)
webapp -> (Camada View)
WEB-INF
web.xml
index.xhtml

pom.xml -> (Gerenciador do Projeto e Dependências)
```

Veja a responsabilidade de cada um desses elementos:

- **src/main/java**: este diretório é o coração da lógica da nossa aplicação, onde todo o nosso código Java residirá. É aqui que implementaremos as camadas ***Model*** e ***Controller***. Para uma boa organização e para seguir as convenções do Java, criaremos subpacotes. Para nosso projeto cadastro-alunos, uma boa estrutura seria: **br.com.cadastroalunos.model** para nossas classes de entidade (o Modelo) e br.com.cadastroalunos.bean para nossos *Backing Beans* (o Controlador).

- **src/main/webapp**: esta pasta representa a raiz da nossa aplicação web e corresponde diretamente à camada ***View***. É aqui que todos os nossos arquivos de interface, como as páginas .xhtml construídas com Facelets, serão armazenados. Dentro dela, o diretório WEB-INF possui um papel especial: os arquivos contidos nele não são acessíveis diretamente pelo navegador, sendo um local seguro para configurações como o arquivo web.xml (o descritor de implantação da aplicação).

  Atenção: é comum que o assistente do IntelliJ crie um arquivo padrão chamado index.jsp. Para aplicações JSF modernas, utilizaremos o **Facelets**, que usa arquivos com a extensão **.xhtml.** Portanto, você pode apagar o arquivo index.jsp. Nós criaremos nosso próprio index.xhtml quando formos implementar a camada View.

- **pom.xml**: embora não seja um diretório, o arquivo pom.xml (*Project Object Model)* é um dos mais importantes do projeto. Ele é a identidade da nossa aplicação para o Maven, responsável por declarar todas as dependências externas (como o JSF, PrimeFaces etc.) e por definir como o projeto deve ser compilado e empacotado.

Com essa estrutura bem definida, temos um mapa claro de onde cada peça da nossa aplicação deve ser colocada, garantindo a separação de responsabilidades que é o pilar da arquitetura MVC. O próximo passo é, finalmente, começar a codificar cada uma dessas camadas.

Implementando as camadas: *model, controller* e *view*

Com o esqueleto do projeto pronto e a função de cada diretório bem definida, é hora de dar vida à nossa aplicação. Agora, vamos codificar os arquivos para cada uma das camadas, começando pelo *Model*, passando pelo *Controller* (nosso *Bean*) e finalizando com a *View* (Burns; Griffin, 2010).

O ***Model*** é a representação dos nossos dados. Para nosso projeto, será uma classe Java simples que representa um aluno. No painel de projeto do IntelliJ, clique com o botão direito na pasta src/main/java e selecione New > Package. Digite br.com.cadastroalunos.model e pressione Enter. Agora, clique com o botão direito no novo pacote model e selecione New > Java Class. Dê o nome de Aluno e cole o seguinte conteúdo:

```
package br.com.cadastroalunos.model;

public class Aluno {
  private String nome;
  private String curso;

  // Getters e Setters são essenciais para que o JSF consiga acessar os dados.
  public String getNome() { return nome; }
  public void setNome(String nome) { this.nome = nome; }
  public String getCurso() { return curso; }
  public void setCurso(String curso) { this.curso = curso; }
}
```

O ***Controller***, que chamaremos de ***AlunoBean***, será o centro da aplicação. Ele irá conter os dados que o formulário manipula e as ações que o usuário pode executar. Então, crie um novo pacote chamado br.com.cadastroalunos.bean. Dentro dele, crie uma nova classe Java chamada AlunoBean com o seguinte código:

```
package br.com.cadastroalunos.bean;
import br.com.cadastroalunos.model.Aluno;
import jakarta.annotation.PostConstruct;
import jakarta.faces.view.ViewScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

// @Named: Torna o bean acessível na View através do nome "alunoBean".
@Named("alunoBean")
// @ViewScoped: O estado do bean (os dados) persiste enquanto o usuário estiver na
mesma página, o que é ideal para formulários com interações.
```

```
@ViewScoped
public class AlunoBean implements Serializable {

  private Aluno aluno = new Aluno();
  private List<Aluno> alunos;

  // @PostConstruct: Este método é chamado automaticamente pelo JSF logo após a
criação do bean, ideal para inicializações.
  @PostConstruct
  public void init() {
    alunos = new ArrayList<>();
  }

  // Ação chamada pelo botão "Salvar" da nossa View.
  public String salvar() {
    // Adiciona o aluno preenchido no formulário à lista.
    alunos.add(aluno);
    // Cria uma nova instância para limpar os campos do formulário para um novo
cadastro.
    aluno = new Aluno();
    return null; // Retornar null mantém o usuário na mesma página.
  }

  // Getters e Setters são essenciais para que a View possa acessar os atributos.
  public Aluno getAluno() { return aluno; }
  public void setAluno(Aluno aluno) { this.aluno = aluno; }
  public List<Aluno> getAlunos() { return alunos; }
  public void setAlunos(List<Aluno> alunos) { this.alunos = alunos; }
}
```

A **View** é a interface com o usuário. Conforme discutido, você deve apagar o index.jsp e, no lugar, criar o arquivo index.xhtml na pasta src/main/webapp. Este arquivo conterá nosso formulário de cadastro e a tabela de listagem. No arquivo index.xhtml, cole o seguinte conteúdo:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:f="http://xmlns.jcp.org/jsf/core">
<h:head>
  <title>Cadastro de Alunos</title>
</h:head>
<h:body>
  <h2>Cadastro de Alunos com JSF</h2>

  <h:form>
     <h:panelGrid columns="2" cellpadding="5">
        Nome: <h:inputText value="#{alunoBean.aluno.nome}" />
        Curso: <h:inputText value="#{alunoBean.aluno.curso}" />
     </h:panelGrid>
     <h:commandButton value="Salvar" action="#{alunoBean.salvar}" />
  </h:form>

 <hr/>

  <h3>Alunos Cadastrados</h3>
  <h:dataTable value="#{alunoBean.alunos}" var="a" border="1" cellpadding="5">
     <h:column>
        <f:facet name="header">Nome do Aluno</f:facet>
        #{a.nome}
     </h:column>
     <h:column>
        <f:facet name="header">Curso</f:facet>
        #{a.curso}
     </h:column>
  </h:dataTable>
</h:body>
</html>
```

Com todas as três camadas do padrão MVC implementadas, o passo final é executar a aplicação no servidor WildFly a partir do IntelliJ IDEA. Após o *deploy*, a IDE deverá abrir seu navegador padrão na URL da aplicação.

É possível que, ao acessar a URL, você se depare com uma tela de erro «Forbidden" (ou 403). Este é um erro de configuração comum e ocorre porque o servidor, por padrão, não reconhece o index.xhtml como a página inicial.

Para corrigir isso, basta informar explicitamente ao servidor qual é o nosso "arquivo de boas-vindas". Para isso, adicione a configuração <welcome-file-list> ao seu arquivo src/main/webapp/WEB-INF/web.xml, como mostrado a seguir:

```
<web-app ...>
  <welcome-file-list>
    <welcome-file> index.xhtml
  </welcome-file>
</welcome-file-list> </web-app>
```

Após adicionar este trecho e reiniciar o servidor, o erro "*Forbidden*" desaparecerá, e a página de cadastro será carregada corretamente. Ao executar a aplicação, você verá uma tela como a ilustrada na Figura 4.

Figura 4 – Aplicação de Cadastro JSF em Execução. / Fonte: o autor.

Descrição da Imagem: a imagem mostra a aplicação de cadastro de alunos rodando em um navegador. No topo, há um formulário com os campos "Nome" e "Curso", e um botão "Salvar". Abaixo do formulário, há uma tabela intitulada "Alunos Cadastrados". A tabela possui duas colunas, "Nome do Aluno" e "Curso". Fim da descrição.

A Figura 4 demonstra o funcionamento da aplicação, com todos os componentes do padrão MVC funcionando em harmonia. A ***View*** (index.xhtml) foi renderizada corretamente, e o fato de os dados aparecerem na tabela confirma que o *Expression Language* (#{...}) está funcionando, conectando a interface ao ***Controller*** (AlunoBean). Este, por sua vez, gerencia a lista de objetos do tipo ***Model*** (Aluno). Ao preencher o formulário e clicar em "Salvar", a tabela é atualizada dinamicamente com o novo aluno (Burns; Griffin, 2010).

Para tornar a aplicação mais robusta, podemos adicionar regras de validação (Burns; Griffin, 2010). Por exemplo, para tornar o campo "Nome" obrigatório, basta uma pequena alteração no arquivo index.xhtml, adicionando o atributo required="true" e a tag <h:message> para exibir o erro.

```
Nome:
<h:inputText id="nome" value="#{alunoBean.aluno.nome}"
    required="true" requiredMessage="O campo Nome é obrigatório." />
<h:message for="nome" style="color: red;" />
```

O resultado dessa validação em ação é ilustrado na Figura 5, onde o usuário tentou salvar o formulário com o campo "Nome" em branco e o JSF automaticamente exibiu a mensagem de erro configurada, impedindo a submissão de dados inválidos.

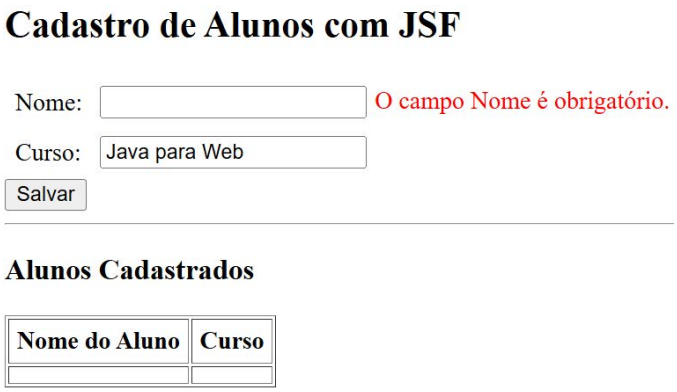


Figura 5 – Exemplo de Validação do JSF em Ação. / Fonte: o autor.

Descrição da Imagem: a imagem exibe a aplicação de cadastro com uma mensagem de validação de erro. O campo "Nome" está vazio e, ao lado, uma mensagem em vermelho informa "O campo Nome é obrigatório.". O campo "Curso" está preenchido com "Java para Web". A tabela de "Alunos Cadastrados" está vazia, indicando que o registro não foi salvo devido ao erro de validação. Fim da descrição.

Com o domínio prático sobre a componentização do JSF e a organização do padrão MVC, você está agora mais preparado para enfrentar os desafios do desenvolvimento de sistemas web corporativos, criando soluções elegantes e de fácil manutenção.

EM FOCO

Para aprofundar ainda mais seu conhecimento, convido você a assistir ao vídeo que preparamos especialmente sobre os fundamentos do JSF e a aplicação do padrão MVC.

NOVOS DESAFIOS

Chegando ao final deste conteúdo, é hora de conectar os conhecimentos teóricos e práticos sobre o JSF com as demandas e oportunidades do mercado de trabalho. Ao longo do estudo, você teve a chance de entender como esse *framework* organiza o desenvolvimento web em Java por meio do padrão MVC, da criação de interfaces com Facelets, do uso de beans controlados pelo CDI, das validações, navegação e até mesmo da integração com bibliotecas como PrimeFaces.

No ambiente profissional, trabalhar com JSF significa integrar-se a projetos robustos e frequentemente corporativos, que prezam por manutenibilidade, reaproveitamento de componentes e compatibilidade com o ecossistema Java EE. A compreensão das estruturas internas do JSF — como o ciclo de vida, escopos e templates — é fundamental para assumir responsabilidades técnicas maiores e desenvolver soluções que atendam critérios de escalabilidade e boas práticas de desenvolvimento.

Agora que você domina os fundamentos, o desafio é colocar esse conhecimento em prática. Que tal tentar construir uma aplicação simples com cadastro, edição e listagem de dados, utilizando tudo o que aprendeu? Ou ainda ir além, explorando recursos complementares como internacionalização, segurança ou persistência com JPA? Essas experiências práticas não apenas consolidam o conteúdo, mas também criam diferenciais importantes em seu portfólio como desenvolvedor.

No mundo real, dominar um *framework* como o JSF exige mais do que conhecer comandos e tags. Exige saber tomar decisões arquiteturais, entender as necessidades do usuário e adaptar a solução às regras do negócio. Que este tema tenha despertado em você não apenas a curiosidade, mas também a confiança para seguir adiante e enfrentar novos desafios no universo do desenvolvimento Java para a web.

ATIVIDADES

1. Uma equipe de desenvolvimento planeja modernizar uma aplicação web legada, atualmente construída com as tecnologias Servlets e JSP. O objetivo é adotar um framework mais recente da plataforma Jakarta EE para a camada de interface, e o JavaServer Faces (JSF) é o escolhido pela equipe.

   Considerando os princípios fundamentais do JSF, qual das seguintes alternativas melhor justifica a escolha do framework para essa modernização?

   a) Porque o JSF promove a separação de responsabilidades com o padrão MVC e o uso de componentes, o que simplifica a manutenção e a reutilização de código.
   b) Porque o JSF aprimora o JSP, permitindo uma mistura mais eficiente de código Java e HTML na mesma página, o que aumenta a produtividade em projetos complexos.
   c) Porque o JSF é um framework *client-side* que executa no navegador, oferecendo uma experiência de usuário mais rica e interativa, similar ao React ou Angular.
   d) Porque o JSF foi projetado para a criação de páginas web estáticas, garantindo melhor performance em comparação com o dinamismo oferecido pelos JSPs.
   e) Porque o JSF elimina a necessidade de um servidor de aplicação como o WildFly, permitindo que as aplicações rodem de forma autônoma em qualquer servidor web simples.

2. Um desenvolvedor está analisando um projeto JSF existente que gerencia perfis de usuários. Ele identifica três artefatos principais:

   1) uma classe User.java que contém apenas atributos como nome e email (com getters e setters);
   2) uma classe UserBean.java que possui a lógica para salvar, buscar e excluir usuários; e
   3) um arquivo profile.xhtml que contém as tags JSF para exibir um formulário e uma tabela de usuários.

   Com base na descrição dos artefatos e na arquitetura MVC, qual é a associação correta entre esses elementos e suas respectivas camadas?

   a) View: profile.xhtml, Controller: UserBean.java, Model: User.java.
   b) View: UserBean.java, Controller: profile.xhtml, Model: User.java.
   c) View: profile.xhtml, Controller: User.java, Model: UserBean.java.
   d) View: User.java, Controller: UserBean.java, Model: profile.xhtml.
   e) View: pom.xml, Controller: User.java, Model: UserBean.jav.a

3. Um estudante, ao finalizar o desenvolvimento do projeto prático cadastro-alunos, executa a aplicação no servidor WildFly. Ao acessar a URL http://localhost:8080/cadastro-alunos--1.0-SNAPSHOT/, em vez de ver o formulário de cadastro, o navegador exibe uma página com a mensagem "Forbidden" (Erro 403). O servidor está rodando e não há erros no console.

   Qual é a causa mais provável para o erro "403 Forbidden" neste cenário específico e como corrigi-lo?

   a) O servidor não está configurado para reconhecer index.xhtml como um arquivo de boas--vindas padrão e, por segurança, proíbe a listagem do diretório. A solução é adicionar uma entrada <welcome-file>index.xhtml</welcome-file> no web.xml.
   b) A dependência do jakarta.enterprise.cdi-api está faltando no pom.xml, impedindo que o Bean seja criado e a página seja acessada. A solução é adicionar a dependência.
   c) O firewall do sistema operacional está bloqueando a porta 8080, impedindo a comunicação entre o navegador e o servidor. A solução é criar uma regra de liberação no firewall.
   d) A anotação @Named("alunoBean") está incorreta no AlunoBean.java, fazendo com que o servidor não encontre o Controller para a página. A solução é corrigir o nome do bean.
   e) O arquivo beans.xml não foi criado na pasta WEB-INF, o que impede a ativação do CDI e causa um erro de acesso. A solução é criar o arquivo beans.xml.

REFERÊNCIAS

ALGAWORKS. **Java EE 7 com JSF, PrimeFaces e CDI**. 2. ed. Belo Horizonte: Algaworks, 2015. Disponível em: https://assets.algaworks.com/ebooks/algaworks-ebook-java-ee-7-com-jsf-primefaces-e-cdi-2a-edicao-20150228.pdf. Acesso em: 17 ago. 2025.

BURNS, E.; GRIFFIN, N. **JavaServer Faces 2.0, the complete reference**. McGraw-Hill, Inc., 2010.

DATHAN, B.; RAMNATH, S. Interactive systems and the model–view–controller architecture. *In*: **Object-oriented analysis, design and implementation**: An integrated approach. Cham: Springer Nature Switzerland, 2025. p. 345-407.

JUNEAU, J. Servlets and JavaServer pages. *In*: **Jakarta EE recipes**: A problem-solution approach. Berkeley, CA: Apress, 2020. p. 1-93.

MANN, K. D. **Java server faces in action**. Dreamtech Press, 2005.

SILVA, E. G. da. **Pro JSF e Ajax**: construindo componentes ricos para a web. 4. ed. Rio de Janeiro: Ciência Moderna, 2007.

GABARITO

1. Alternativa A.

   A) Correto. Esta alternativa descreve com precisão o principal benefício do JSF. O framework foi projetado para organizar o código, separar as responsabilidades (lógica de negócio, controle e apresentação) e incentivar a reutilização por meio de componentes, facilitando a manutenção.
   B) Incorreto. O JSF foi criado justamente para resolver o problema da mistura de Java e HTML, e não para aprimorá-la. A tecnologia de visualização do JSF, o Facelets (.xhtml), desencoraja essa prática, que era comum e problemática nos JSPs.
   C) Incorreto. O JSF é um framework server-side. A lógica dos componentes e o ciclo de vida da aplicação rodam no servidor, que então envia o HTML resultante para o navegador.
   D) Incorreto. O JSF é, por definição, um framework para o desenvolvimento de interfaces web dinâmicas e aplicações corporativas complexas, e não estáticas.
   E) Incorreto. O JSF, como parte da especificação Jakarta EE, depende de um ambiente de execução que implemente as APIs de Servlet e outras, como um servidor de aplicação (WildFly) ou um contêiner web (Tomcat).

2. Alternativa A.

   A) Correto. Esta alternativa mapeia perfeitamente as responsabilidades descritas: o arquivo .xhtml é a interface visual (View) ; a classe UserBean.java, que contém a lógica de salvar e buscar, é o intermediário que processa as ações (Controller) ; e a classe User.java, que apenas estrutura os dados, é o Model.
   B) Incorreto. Esta opção inverte os papéis da View (que é a camada de apresentação, .xhtml) e do Controller (a classe com a lógica de ações, UserBean.java).
   C) Incorreto. A classe User.java representa a estrutura dos dados (Model), e não a lógica de controle (Controller). O Controller é a classe que executa as ações, neste caso, o UserBean.java.
   D) Incorreto. A camada View é a interface visual (.xhtml), e não a classe de dados (User.java). A classe de dados é o Model.
   E) Incorreto. O pom.xml é o arquivo de configuração e gerenciamento de dependências do projeto (via Maven), e não faz parte diretamente das camadas da arquitetura MVC.

3. Alternativa E.

E) Correto. O erro 403 ao acessar a raiz de uma aplicação web quase sempre indica que a listagem de diretórios está desabilitada e o servidor não encontrou um arquivo padrão (welcome-file) para exibir. A solução padrão do Jakarta EE é configurar a <welcome-file-list> no web.xml para apontar para o index.xhtml.
A) Incorreto. A falta da dependência do CDI causaria problemas na renderização da página (o Expression Language não funcionaria) ou um erro 500 (Erro Interno do Servidor), mas não um erro 403 (Forbidden).
B) Incorreto. Se o firewall estivesse bloqueando a porta, o navegador não conseguiria se conectar ao servidor e exibiria um erro de "não é possível acessar esse site", e não uma resposta "Forbidden" vinda do próprio servidor.
C) Incorreto. Um erro na anotação do bean normalmente resultaria em um erro 500 na hora de renderizar a página, pois o Expression Language #{alunoBean} falharia, mas não um 403.
D) Incorreto. A ausência do beans.xml causaria sintomas similares à falta da dependência do CDI, impedindo o processamento correto da página, mas não um erro de acesso proibido ao diretório.